Eötvös Loránd University

Faculty of Informatics

Dept. of Software Technology and Methodology

Detecting and Explaining Cryptographic Misuse in C/C++ via LLVM/Clang: A Taxonomy-Driven Integration into CodeChecker

Supervisor:
Zoltán Porkoláb
Associate Professor

Author:
Chandler Camarena
Computer Science MSc

Budapest, 2026

Contents

1 Introduction **5**

1.1 Research Questions 6

1.2 Summary of Contributions 7

1.3 Thesis Structure 8

2 Theoretical Foundations **9**

2.1 Object Representation in C and C++ 9

2.2 Alignment and ABI 10

2.2.1 ABI Evidence of Representation Artifacts 10

2.3 Standard Terminology 10

2.4 Indeterminacy and Initialization 11

2.5 From Language Rules to ABI 12

3 Related Work **13**

3.1 Representation-Level Security Properties 13

3.2 Security Invariants 14

3.3 Static Structural Analysis 15

3.4 Trust Boundaries and Information Flow 16

3.5 Cryptographic Misuse Detection 17

3.6 Compiler-Based Security Mechanisms 18

3.7 Synthesis and Positioning 18

4 Security Misconfiguration as a Semantic Class **20**

4.1 Motivation 20

4.2 Program Model 20

4.2.1 Boundary Contexts and Observation 21

4.3 Security Invariants 24

4.4 Definition of Semantic Misconfiguration 25

4.5 Trust-Boundary Leakage 25
4.6 Layered Semantic Model 26

5 Taxonomy-Driven Integration 28
5.1 Taxonomy Requirements 29
5.2 Top-Level Taxonomy 29
5.2.1 Taxonomy Entry Schema 30
5.2.2 Example Entry: Aggregate Authentication 31
5.2.3 Taxonomy Mapping 32
5.3 Cryptographic Misuse Families 33
5.3.1 CM: Cryptographic Misuse Families 34
5.4 Trust-Boundary Misconfigurations 35
5.4.1 TB: Trust-Boundary Misconfiguration Families 35
5.5 Data-Movement Misconfigurations 36
5.5.1 DM: Representation Reads 36
5.6 Evidence Lattice 39
5.6.1 Evidence Domain and Partial Order 39
5.6.2 Evidence Levels 40
5.6.3 Case-Study Instantiation 41
5.6.4 Evidence Facts 41

6 Modeling Framework 43
6.1 Model Interface 43
6.2 Boundary Transfer Events 44
6.3 Object Representation and Padding 44
6.4 Initialization Approximation 45
6.5 Leakage Predicate 46
6.6 Worked Examples 47
6.6.1 Example 1: Field-wise Initialization 47
6.6.2 Example 2: Whole-Object Initialization 47
6.7 Modeling Assumptions 48
6.7.1 Assumptions 48
6.7.2 Staged Extensions 48

7 Implementation as a CodeChecker Extension 49

7.1 Integration Architecture 49
7.2 Boundary Annotations 50
7.3 Event Extraction 50
7.4 Type Selection 50
7.5 Padding Computation 51
7.6 Initialization Heuristics 51
7.7 Leak Evaluation 52
7.8 Diagnostics and Reporting 52
7.9 Implementation Scope 53

8 Evaluation **54**
8.1 Research Questions 54
8.2 Dataset Selection 55
8.2.1 Selection Criteria 55
8.2.2 Selected Projects 56
8.3 Boundary Annotation Protocol 56
8.4 Measurement Plan 57
8.4.1 Metrics 57
8.4.2 Data Collection 58
8.5 Synthetic Benchmark Validation 58
8.6 Empirical Results 59
8.6.1 Per-Project Analysis 60
8.6.2 Manual Validation of Findings and Suppressions 62
8.6.3 Cross-Project Observations 63
8.7 Performance 64
8.8 Threats to Validity 65
8.9 Summary 67

9 Limitations and Future Work **68**
9.1 Known Limitations of the Current Prototype 68
9.2 Automatic Boundary Inference 69
9.3 Interprocedural Initialization Analysis 69
9.4 Pointer-Based Transfers 70
9.5 Configuration-Level Semantic Misconfigurations 70
9.6 C++ Object Semantics 71

9.7 Empirical Scaling 71
9.8 Formal Soundness 72
9.9 Extending Beyond Representation-Level Leakage 72
9.10 Returning to Cryptographic Misuse Families 72

10 Conclusion **73**

A Synthetic Benchmark Suite **75**
A.1 Benchmark Matrix 75
A.2 Purpose of the Suite 75

Bibliography **77**

Chapter 1

Introduction

Cryptographic libraries are routinely misused in real-world software systems: developers select deprecated algorithms, disable authentication, reuse nonces or IVs, omit required checks, or apply APIs with subtly incorrect parameterization [1, 2, 3, 4]. These failures are rarely "one bad line" in isolation; rather, they arise from *semantic and configuration-level mismatches* between the security guarantees assumed by the programmer and the guarantees actually delivered by the program. Because many of these mismatches are reflected in types, call patterns, object lifetimes, and compilation-time structure, they are, at least in part, *visible to compiler infrastructure* [5, 3, 4].

These observations motivate not only a compiler-centric approach to cryptographic misuse detection, but also a broader methodology for security-relevant misconfigurations whose operational manifestation is visible in compiler semantics. This thesis presents a taxonomy-driven framework that leverages LLVM/Clang analyses to detect and explain security-relevant misconfigurations in C/C++, and integrates the resulting checks into CodeChecker [6, 7]. Concretely, we pursue a methodology in which security-relevant misconfigurations are (i) organized in a taxonomy that reflects *what the compiler can observe*, (ii) detected using LLVM/Clang-based analysis, and (iii) reported with explanations and safer alternatives that are actionable for developers.

The thesis begins from the cryptographic-misuse literature, but studies a broader question: whether selected security semantic misconfigurations can be modeled as compiler-visible properties. Among the possible families in that broader space, the present work concentrates on a concrete and deliberately scoped instantiation:

representation-level leakage across explicitly annotated trust boundaries in C/C++.

In modern systems, compartmentalization and least-privilege decomposition turn previously internal interactions into *trust-boundary transfers*: values and object representations cross between components with different confidentiality assumptions [8]. In C and C++, this raises a particularly sharp hazard that is both (a) easy to trigger in practice and (b) naturally expressible in compiler terms: *representation-level information leakage* caused by transferring composite objects whose *object representation* includes compiler-introduced padding bytes [9, 10, 11, 12, 13]. Even when every named field is "logically initialized," padding bytes may retain residual values; copying or transmitting the object across an explicitly annotated boundary can therefore disclose unintended information without contradicting functional correctness [11, 14, 15].

This thesis investigates whether such misconfigurations can be modeled as *compiler-visible semantic properties* and detected with a practical static checker integrated into CodeChecker [7]. We focus on a controlled setting in which trust boundaries are expressed via explicit source-level annotations. Boundary inference and whole-program flow sensitivity are treated as out-of-scope extensions, allowing the evaluation to isolate what can be achieved with a primarily AST-driven analysis that combines type- and layout-aware reasoning with boundary-policy annotations and call-effect modeling for boundary-transfer operations. The resulting evaluation is therefore designed as a contrastive characterization study rather than a prevalence survey: it asks where the modeled event surface appears, how API design conditions that surface, and how the evidence lattice behaves on representative real-world libraries.

1.1 Research Questions

This thesis addresses the following research questions:

1. Can security misconfigurations in C/C++ be modeled as compiler-visible semantic properties suitable for taxonomy-driven static checking, and how does representation-level leakage across trust boundaries instantiate this idea?

2. To what extent can AST-level analysis in LLVM/Clang detect boundary-violating transfers that expose object-representation bytes (e.g., padding) un-

der explicitly annotated trust-boundary assumptions?

3. What are the precision and scalability tradeoffs of detecting such structural misconfigurations without interprocedural flow sensitivity, and what classes of cases require richer program reasoning?

1.2 Summary of Contributions

This thesis makes the following contributions:

1. A framing of security semantic misconfiguration as a compiler-visible class of properties, motivated by cryptographic misuse and instantiated through the concrete case study of representation-level trust-boundary leakage in C/C++.

2. A modeling framework that treats explicitly annotated trust boundaries as semantic contexts within which object layout and object representation become security-relevant, together with a confidentiality invariant stated over observable representations.

3. A taxonomy and evidence discipline that supports implementable, explainable checking by distinguishing byte-defined observations, representation artifacts, and initialization evidence.

4. The design and implementation of a Clang/CodeChecker static-analysis prototype that identifies candidate boundary-transfer events likely to expose object-representation bytes using primarily AST-level reasoning.

5. A contrastive empirical evaluation, combining a synthetic benchmark suite with representative real-world C libraries, that characterizes the modeled event surface, validates emitted findings manually, and shows how API design and initialization discipline shape observed outcomes.

6. A discussion of limitations and future directions, including boundary inference, interprocedural initialization reasoning, and extension of the same taxonomy-driven methodology to additional misconfiguration families such as cryptographic misuse patterns.

1.3 Thesis Structure

Chapter 2 introduces theoretical foundations, including object representation semantics, alignment and padding behavior, and the definition of trust boundaries. Chapter 3 reviews related work. Chapter 4 defines security misconfiguration as a semantic class. Chapter 5 presents the taxonomy-driven integration. Chapters 6 and 7 present the modeling framework and its implementation in LLVM/Clang. Chapter 8 evaluates the prototype on synthetic benchmarks and representative real-world C libraries. Chapter 9 discusses limitations and future research directions. Chapter 10 concludes.

Chapter 2

Theoretical Foundations

2.1 Object Representation in C and C++

The C and C++ language standards distinguish between the *value representation* and the *object representation* of a type [9, 16]. The object representation of an object consists of the sequence of bytes that occupy its storage [9, 16]. For scalar types, the value representation typically coincides with the object representation. For aggregate types such as structures and classes, however, the object representation may include additional bytes introduced to satisfy alignment and layout requirements [9, 16].

The C standard permits implementations to insert padding bytes between structure members or at the end of a structure in order to satisfy alignment constraints [9]. Similarly, the C++ standard allows padding within class objects to preserve layout requirements and maintain ABI compatibility [16]. These padding bytes are part of the object representation but do not correspond to any declared data members [9, 16].

Unless program code explicitly writes the full storage, padding bytes need not be semantically determined by member-wise initialization alone. As a result, even when all declared fields are assigned values, the full object representation may still contain residual or indeterminate bytes [9, 11, 14]. This distinction between declared fields and complete object representation forms the basis for structural information leakage across trust boundaries, a defect class recognized in both empirical security work and industrial static-analysis tooling [11, 12, 13].

2.2 Alignment and ABI

Alignment requirements are determined by both language rules and the target Application Binary Interface (ABI) [9, 16, 10]. Compilers insert padding to ensure that each member of a structure satisfies its alignment constraints. For example, a structure containing a one-byte character followed by a four-byte integer may include padding between the members to align the integer on a four-byte boundary [10].

2.2.1 ABI Evidence of Representation Artifacts

The ISO language standards define where padding *may* appear, while the platform ABI determines where padding *does* appear in concrete layouts and interfaces [9, 16, 10]. The System V AMD64 ABI makes these representation artifacts explicit. For example, it specifies that `long double` occupies 16 bytes of storage but only the first 10 bytes are significant, leaving six bytes of tail padding whose contents are undefined [10]. This is a direct instance of "non-semantic" bytes within an object representation that may nevertheless be copied or exposed [10].

The same ABI also elevates alignment to an interface-level constraint. In its aggregate classification procedure for argument passing, an aggregate is passed in the `MEMORY` class if it is large enough *or if it contains unaligned fields* [10]. Consequently, call and return boundaries can force materialization and copying of aggregate object representations in ways that are both ABI-mandated and security-relevant [10, 11, 15]. This motivates treating by-value calls/returns as canonical trust-boundary transfer events in the modeling framework.

2.3 Standard Terminology

To avoid ambiguity, this thesis uses ISO/IEC 9899:2018 (C18) terminology when discussing representation-level behavior in C [9]. The following terms are particularly load-bearing for trust-boundary leakage analysis:

Undefined behavior (C18 §3.4.3) Program behavior for which the standard imposes no requirements on the outcome [9].

Object representation (C18 §6.2.6.1) The sequence of bytes that occupies an object in memory; formally, objects are composed of a contiguous sequence

of one or more bytes [9]. For aggregate types, the object representation can include bytes that are not associated with any declared member.

Value representation (C18 §6.2.6) The subset of the object representation that participates in representing the value of the object [9]. For many scalar types, value and object representation coincide; for aggregates, they need not.

Padding within structures (C18 §6.7.2.1) Implementations may insert unnamed padding *within* a structure object and at its end to satisfy alignment and layout constraints, but not at its beginning [9]. These padding bytes are part of the object representation yet do not correspond to declared members.

Unspecified value (Clause 3 terminology) A value chosen by the implementation from a set of possibilities permitted by the standard, with no additional guarantees as to which one is selected [9].

Indeterminate value (Clause 3 terminology; see also C18 §6.2.6.1) A state in which an object does not have a determinate semantic value; depending on type and context, this may behave as an unspecified value or may involve a trap representation [9].

Trap representation (Clause 3 terminology) A bit-pattern for a type that, when read as a value of that type, can cause undefined behavior [9]. This thesis treats trap representations as a worst-case endpoint of indeterminacy when reasoning about security and reliability.

This clause-anchored vocabulary is essential for distinguishing (i) standards-permitted bytewise copying of an object representation from (ii) potentially undefined evaluation of an indeterminate value as a typed value [9].

2.4 Indeterminacy and Initialization

Two standard mechanisms explain why padding bytes can retain stale data even in otherwise "initialized" aggregates:

- **Uninitialized automatic objects (C18 §6.7.9).** Objects with automatic storage duration that are not explicitly initialized can begin with indeterminate values or bytes, depending on type and context [9].

- **Aggregate initialization is member-directed (C18 §6.7.9).** Aggregate initialization is defined in terms of initializing the declared members; unnamed padding within the object representation is not thereby guaranteed to be overwritten as part of that process [9]. Consequently, padding bytes may remain residual or indeterminate even when every named field is assigned a determinate value [9, 11, 14].

The security implication is subtle but critical: *bytewise transfer of an aggregate across a boundary can copy the entire object representation, including padding, without any explicit program action to "read" padding as a typed value.* The result can still leak historical data (e.g., stack residues) to a lower-trust context [11, 14, 15, 12, 13].

2.5 From Language Rules to ABI

Although the C standard specifies where padding *may* occur, the concrete placement and amount of padding is determined by the target ABI and compiler decisions (e.g., member alignment, overall object alignment, and tail padding to preserve array element alignment) [9, 10]. This thesis relies on Clang's semantic model and layout computation to obtain these details statically, bridging the gap between abstract standard permissions and concrete compiled layouts [6].

Chapter 3

Related Work

This chapter surveys research relevant to the theoretical framework and implementation developed in this thesis. Rather than organizing prior work by institutional origin or tool category, we examine it along conceptual dimensions that bear on the modeling and analysis approach: representation-level security properties, invariant-based reasoning, structural static analysis, trust-boundary modeling, cryptographic misuse detection, and compiler-based security mechanisms.

3.1 Representation-Level Security Properties

Object representation and padding semantics have long been discussed in systems programming and secure coding literature. The ISO C and C++ standards define the distinction between value representation and object representation, permitting padding bytes within aggregate types [9, 16]. ABI specifications such as the System V AMD64 ABI and the Itanium C++ ABI further determine concrete layout, alignment, and calling-convention behavior [10, 17]. These documents establish that padding is a permitted and implementation-defined aspect of representation semantics.

Secure coding guidelines, including CERT DCL39-C and DCL55-CPP, explicitly warn against passing structures or class objects across trust boundaries without ensuring full initialization [18, 19]. These guidelines recognize that padding bytes may contain residual memory values and may become observable during copying or serialization. The same defect class is explicitly recognized in industrial static-

analysis tooling (e.g., padding-byte information disclosure checks) and is typically remediated by whole-object initialization or explicit serialization [12, 13].

Beyond standards and guidelines, prior security research has demonstrated that representation-level leakage is not merely a theoretical concern. Cho et al. show that uses of uninitialized stack variables in the Linux kernel can become exploitable information leaks, including cases where structure padding is disclosed through interface-oriented copying patterns [11]. Lu et al. propose UniSan as a compiler-based mitigation against uninitialized-memory disclosure, further underscoring that residual representation bytes are a practical security problem and a plausible target for compiler-assisted enforcement [14]. Österlund et al. similarly treat kernel information leakage as an observable security phenomenon and detect such leaks through execution-level divergence, reinforcing the idea that confidentiality violations can arise from bytes that are semantically "outside" the intended logical state [15].

However, these discussions are typically framed either as secure programming advice, empirical vulnerability analysis, or mitigation engineering rather than as formal semantic properties. The phenomenon is described operationally rather than modeled as a predicate over representation semantics $R(P)$ and boundary contexts B. This thesis builds upon these observations by formalizing trust-boundary leakage as a structural invariant violation rather than as an isolated coding guideline or defect pattern.

3.2 Security Invariants

Formal methods research frequently models security properties as invariants over program execution. Information-flow control systems define noninterference properties that constrain how sensitive data propagates through program states [20, 21, 22]. Type systems and language-based enforcement mechanisms have been developed to encode and statically enforce such invariants [23, 24].

These approaches typically reason over control-flow, data-flow, and label propagation across program executions. In contrast, the invariant $I(P)$ defined in Chapter 4 is structural and representation-oriented. It constrains observable object representations at trust boundaries rather than modeling dynamic propagation of labeled data.

The distinction is not one of expressive power but of modeling level. Information-flow systems reason about dynamic dependencies and noninterference across executions; this thesis investigates whether structural properties available within compiler-visible semantics can approximate a restricted confidentiality invariant without full flow sensitivity [20, 22, 23].

3.3 Static Structural Analysis

Static analysis tools such as Clang-based analyzers, CodeQL, Coverity-style checkers, and research prototypes perform pattern-based and semantic checks over source code and intermediate representations [5, 25]. Many such tools rely on AST-level inspection combined with rule-based matching or lightweight data-flow reasoning, while others build richer abstract models for selected vulnerability classes [26, 5].

Clang-based analysis frameworks provide access to complete type definitions, layout metadata, source annotations, and compiler-derived semantic information [27, 6]. These capabilities enable detection of syntactic patterns and structural properties without executing the program. In addition, CodeChecker provides a practical integration layer for Clang-based analyses in developer workflows, including diagnostic surfacing and triage [7].

Prior work in static analysis has extensively explored tradeoffs between flow-sensitive and flow-insensitive reasoning [25, 5, 26]. Flow-sensitive approaches improve precision but increase computational complexity. Pattern-based and structurally anchored approaches scale well and are often easier to explain, but may lack completeness when security properties depend on long-range dataflow, aliasing, or sequencing [5, 26].

The modeling approach in this thesis aligns with AST-level structural reasoning. Rather than attempting whole-program information-flow tracking, it evaluates whether representation semantics $R(P)$ combined with explicit boundary annotations B are sufficient to approximate violations of invariant $I(P)$. In this sense, the work is closer to rule-driven, explanation-oriented static checking than to whole-program soundness claims [4, 7].

3.4 Trust Boundaries and Information Flow

Trust boundaries are commonly modeled in security research as interfaces across which confidentiality or integrity assumptions change. Recent empirical work on compartmentalized systems emphasizes that the interface itself becomes a primary attack surface once code is split into compartments. Lefeuvre et al. define *Compartment Interface Vulnerabilities* (CIVs) and demonstrate that such vulnerabilities can negate isolation benefits across many compartmentalization scenarios; importantly for this thesis, their CIV-aware interface guidelines explicitly warn against sharing uninitialized data across interfaces and note that compiler-introduced padding can remain uninitialized even when logical fields are validated [8].

In dynamic taint analysis, confidentiality-relevant data is modeled using labels introduced at *sources*, propagated through program operations, and checked at *sinks*. Concretely, a taint analysis defines: (i) a set of sources (e.g., network input, IPC messages, untrusted buffers), (ii) a propagation relation over program semantics (explicit data dependencies and, in some systems, selected implicit/control dependencies), and (iii) a set of sinks (e.g., file writes, network sends, logging, cryptographic APIs) where tainted values indicate potential policy violations. This "sources–propagation–sinks" model is widely used in dynamic systems such as TaintCheck and TaintDroid to track cross-boundary influence during execution [28, 29]. Static information-flow systems similarly reason about boundary transitions, but typically require interprocedural flow and alias modeling to approximate propagation soundly [22, 23, 24].

Many existing approaches attempt to infer boundaries implicitly through privilege levels, network interfaces, or API calls. These systems typically require interprocedural analysis and sophisticated alias modeling. In contrast, this thesis adopts an explicit annotation-based model of trust boundaries. Boundary contexts B are defined at source level, isolating representation-level reasoning from boundary inference complexity. This modeling choice allows focused evaluation of compiler-visible structural semantics while acknowledging that boundary inference constitutes a separate research problem [8, 28, 29].

3.5 Cryptographic Misuse Detection

A second major line of related work concerns the detection of cryptographic misuse. Empirical studies have repeatedly shown that security failures often arise not from broken primitives, but from incorrect API usage, missing validation, weak parameters, nonce/IV misuse, or protocol-level omissions [1, 2, 30]. This literature is important for the present thesis because it demonstrates that security-relevant defects frequently manifest as *misconfigurations of otherwise valid code*, rather than as memory-safety errors or explicit undefined behavior.

Egele et al.'s CryptoLint showed that large numbers of Android applications violated cryptographic best practices through misuse patterns that were detectable statically [1]. Rahaman et al.'s CryptoGuard pushed this direction toward higher precision, emphasizing both scalability and reduction of false positives in misuse reporting [2]. Krüger et al.'s CrySL/CogniCrypt line of work made this even more explicit by treating cryptographic API usage constraints as declarative rules that can drive static checking [3]. More recently, Chen et al. focused directly on precise reporting of cryptographic misuses, highlighting that actionable diagnostics require careful alignment between evidence, explanation, and developer-facing messages [4].

This literature strongly informs the taxonomy-driven methodology used in this thesis. In particular, it supports the view that a static checker should not merely detect potentially dangerous patterns; it should also encode what evidence is available, what security property is at stake, and what remediation is appropriate. That perspective directly motivates the evidence-oriented reporting discipline introduced in Chapter 5 [2, 4]. At the same time, the cryptographic-misuse literature usually focuses on API-level misuse and protocol constraints, whereas this thesis instantiates the broader methodology on a representation-level confidentiality problem tied to layout, padding, and boundary transfer semantics [1, 3, 4].

Related work on API misuse in C projects also reinforces the relevance of this framing. Gu et al. study API-misuse bugs in open-source C projects, supporting the broader claim that API misuse is a meaningful defect class even outside managed-language ecosystems [31]. Survey work on misuse of security APIs further shows that the field has converged on misuse detection as a distinct research problem spanning empirical studies, static analyses, and developer-oriented diagnostics [30]. Repair-oriented systems such as CDRep are also relevant as evidence that detection and

remediation can be coupled, even though the present thesis focuses on detection and explanation rather than automatic repair [32].

3.6 Compiler-Based Security Mechanisms

Modern compilers and toolchains provide security hardening mechanisms such as stack protectors, position-independent code generation, and related mitigation support. These mechanisms alter code generation or linking behavior while preserving language-level correctness.

Compiler flags controlling such features represent configuration-level security decisions. Disabling stack protection or related hardening does not introduce syntax errors or undefined behavior, yet may weaken system security posture.

These examples illustrate that representation and configuration semantics influence security properties without violating language rules. While this thesis focuses on structural trust-boundary leakage rather than compiler hardening flags, both fall within the broader category of representation-induced or configuration-induced invariant violations as defined in Chapter 4.

3.7 Synthesis and Positioning

The reviewed literature demonstrates that representation-level leakage, invariant-based reasoning, static structural analysis, trust-boundary modeling, cryptographic misuse detection, and compiler security mechanisms are all established areas of study. However, these areas are typically treated independently [5, 8, 2].

Secure coding standards describe padding leakage operationally but do not formalize it as a semantic invariant over representation semantics [18, 19, 12, 13]. Information-flow systems enforce confidentiality dynamically or via label-oriented static reasoning, but do not isolate representation-induced structural violations as a first-class compiler-visible defect family [22, 23, 24]. Static analysis tools provide structural inspection but often focus on memory safety, control-flow errors, or API misuse rather than object-representation leakage at annotated trust boundaries [5, 25, 31]. Compiler hardening research addresses configuration-level defenses rather than representation-level invariants.

This thesis occupies the intersection of these dimensions. It formalizes trust-boundary information leakage as a structural violation of a confidentiality invariant $I(P)$, grounded in representation semantics $R(P)$ and explicitly annotated boundary contexts B. It then evaluates whether AST-level compiler-visible semantics are sufficient to approximate such violations without full interprocedural flow tracking.

The objective is not to replace dynamic information-flow systems nor to compete with comprehensive static analyzers. Rather, it is to investigate the expressive limits of structural semantic reasoning within the compiler as a vehicle for modeling selected classes of security semantic misconfiguration.

Chapter 4

Security Misconfiguration as a Semantic Class

4.1 Motivation

Traditional vulnerability analysis primarily focuses on violations of language rules, memory safety errors, or incorrect control-flow behavior. These include buffer overflows, use-after-free errors, and other forms of undefined behavior [5, 25]. In contrast, this thesis investigates a class of weaknesses that arise despite syntactic correctness and standards compliance. These weaknesses emerge from permitted representation or configuration semantics that weaken system-level security guarantees without invoking undefined behavior [8, 11, 12, 13].

To reason precisely about this class of weaknesses, we introduce a formal definition of *security semantic misconfiguration*. This definition separates language-level correctness from security invariants and enables structured modeling of representation-induced confidentiality violations.

4.2 Program Model

Let P denote a program written in C or C++.

We assume:

- P is syntactically well-formed.
- P satisfies the semantic rules of the ISO C or C++ standard [9, 16].

- P does not invoke undefined behavior UB.

Let:

- $L(P)$ denote the language-level semantics of P.
- $R(P)$ denote the object representation semantics of P, including layout, alignment, and padding as determined by the compiler and target ABI [9, 16, 10].
- $B \subseteq \mathcal{C}$ denote a set of explicitly annotated trust-boundary contexts within the program.

We separate three notions. Language-level semantics $L(P)$ govern correctness under the programming standard. Representation semantics $R(P)$ govern how objects are realized in memory, including their object representations and any permitted padding within aggregates [9, 16]. Trust-boundary contexts B denote program points where these byte-level representations may become observable across domains with different confidentiality assumptions [8, 18, 19].

Threat model and interpretation of annotations. Boundary annotations in this thesis encode a *confidentiality partition*: a directed pair $(d_s \to d_t)$ indicates that data originating in d_s must not cause additional information to become observable in d_t beyond what the interface contract intends to disclose. The analysis is therefore *confidentiality-first*; integrity and availability concerns (e.g., corrupted sizes, TOCTOU, protocol ordering) are represented in the broader taxonomy but are not the primary semantic guarantee instantiated by the prototype. Domains are directed rather than symmetric: $(d_s \to d_t)$ and $(d_t \to d_s)$ may have different policies, reflecting common designs where trusted code sends a constrained view to an untrusted component. Two regions have different confidentiality assumptions if an attacker in d_t is permitted to observe interface-level bytes and outputs, while internal state in d_s is not [20, 22, 8].

4.2.1 Boundary Contexts and Observation

Boundary contexts. A *trust-boundary context* is an explicit partition of program execution into regions whose confidentiality assumptions differ. Formally, let $\mathcal{D}$ be a

finite set of *domains* (e.g., `trusted`, `untrusted`) and let dom : Loc $\to \mathcal{D}$ map program locations to their domain. A boundary context B is a set of *directed boundary interfaces*:

$$B \subseteq \mathcal{D} \times \mathcal{I} \times \mathcal{D},$$

where $\mathcal{I}$ is the set of interface sites (functions, call sites, or externally visible API boundaries) selected by annotations. An element $(d_s, i, d_t) \in B$ denotes that interface i transfers data from a source domain d_s to a target domain d_t.

Why boundaries are explicit. Without annotations, boundary inference is ambiguous in common codebases because the same function may be used both as an internal helper and as an externally reachable interface. The following minimal example illustrates the ambiguity:

```
struct Msg { uint8_t t; uint64_t x; uint32_t n; };

void send_msg(struct Msg m);  // sometimes internal, sometimes exported

void internal_path(void) {
  struct Msg m; m.t=1; m.x=42; m.n=7;
  send_msg(m);                 // intra-domain: not a security boundary
}

void exported_path(void) {
  struct Msg m; m.t=1; m.x=42; m.n=7;
  send_msg(m);                 // cross-domain: boundary observation is security
}
```

A static analysis that attempts to infer whether `send_msg` is a trust boundary from syntax alone will either over-report (treating all calls as boundary transfers) or under-report (missing externally relevant call sites). Explicit annotations encode the intended trust partition, allowing the analysis to remain precise and explainable at the boundary definition itself [8].

Boundary-transfer events. For a fixed B, we write E_B for the set of *boundary-transfer events* induced by executing P in the presence of the interface set B. In the

case study implemented in this thesis, events are restricted to C/C++ call/return transfers that copy record objects by value:

- **Call-by-value:** passing an argument of record type T by value across a boundary interface i.
- **Return-by-value:** returning a record object of type T by value from a boundary interface i.

Each event $e \in E_B$ is associated with (i) an interface i, (ii) a direction $(d_s \to d_t)$, (iii) a concrete object instance o of record type T, and (iv) a byte string equal to the object representation $\mathrm{Bytes}(o)$ at the event point [9, 10].

Cross-domain observation. We say that an event $e = (d_s, i, d_t, o, \mathrm{Bytes}(o))$ induces a *cross-domain observation* if the target domain d_t can distinguish executions based on $\mathrm{Bytes}(o)$ at the boundary. Operationally, in the by-value case study, the observation consists of the exact byte sequence copied by the call/return ABI for the transferred object. In other words, observation is defined over *object representations* rather than abstract field values: if two executions produce different $\mathrm{Bytes}(o)$ at the boundary, then the target domain may receive or record different bytes, even when the source code never interprets those bytes as typed values [10, 11, 15, 18, 19].

Proposition 4.1 (By-value representation leakage under partial initialization). Let $e \in E_B$ be a call-by-value or return-by-value boundary-transfer event that copies an object o of record type T from domain d_s to d_t. Suppose: (i) the ABI layout for T contains at least one padding byte; (ii) the program does not perform a dominating whole-object initialization of o prior to e; and (iii) at least one named field of o is assigned so that the transfer is nontrivial. Then there exists an execution consistent with the C abstract machine in which the cross-domain observation induced by e depends on indeterminate bytes in the object representation of o.

Informal argument. Padding bytes are part of the object representation of a record object, while the C abstract semantics does not require assignments to named fields to assign any particular values to padding [9]. Under (i), $\mathrm{Bytes}(o)$ contains padding positions. Under (ii), the program does not establish fixed values for all bytes of o before the transfer. Consequently, there exist executions in which the padding bytes retain indeterminate or prior stack contents [9, 11, 14]. Since the ABI transfer for by-value arguments/returns copies the full representation of o as bytes, the boundary

observation includes those padding bytes [10]. Therefore, the observation can vary with indeterminate padding even when all named fields have well-defined values [11, 15]. □

4.3 Security Invariants

Observability Lemma (informal). In this model, the phrase "byte-level representations may become observable across domains with different confidentiality assumptions." Many boundary mechanisms treat memory as an unstructured byte string, so the *entire object representation* (including padding) can influence what a lower-trust domain receives or records, even when no code ever interprets those bytes as typed values. Concretely, observability arises whenever the program performs a *boundary-transfer operation that reads an object representation* at a sink whose interface is defined over bytes rather than fields [25, 28]. Canonical instances, which are used repeatedly in the checker taxonomy, include: (i) *file / IPC / network writes*, where `write`/`fwrite`, pipes/message queues, and socket sends transmit exactly the supplied byte sequence into an external or differently trusted recipient; (ii) *raw comparison and emission*, such as `memcmp(&a,&b,sizeof a)` and binary logging/hexdump utilities, where representation bytes are compared or emitted verbatim, and thus can leak through control decisions or diagnostic channels; and more generally, any API that consumes raw struct bytes rather than fieldwise serialization [11, 8, 18, 19, 12, 13]. These sinks justify modeling trust-boundary contexts as points where representation semantics $R(P)$ can determine cross-domain observations, independently of language-level correctness or UB (undefined behaviour). Accordingly, we define violations in terms of observable effects at these representation sinks, and we use this notion to instantiate the boundary contexts B and events E_B.

Let $I(P)$ denote the confidentiality invariant for program P. Informally, $I(P)$ holds iff no data outside the logically initialized state of an object becomes observable across trust boundaries. Equivalently, for every boundary event $e \in E_B$ involving an object o, the observation induced by e may depend only on the intended logical field values of o (not on padding or stale bytes in its representation).

This invariant is not derived from language correctness but from system-level security requirements. A program may satisfy $L(P)$ and still violate $I(P)$. In this sense, the model is adjacent to language-based information-flow security, but nar-

rower in scope: it targets confidentiality violations induced by transferring object representations across explicitly annotated boundaries rather than arbitrary secret-dependent computation [20, 23, 22].

4.4 Definition of Semantic Misconfiguration

We define a security semantic misconfiguration as follows:

$$\text{SemanticMisconfig}(P) \iff \Big(\text{WellFormed}(P) \wedge \neg\text{UB}(P)\Big) \wedge \neg I(P)$$

In words:

A security semantic misconfiguration occurs when a program remains syntactically correct and standards-compliant, yet violates a security invariant due to representation or configuration semantics.

This definition distinguishes semantic misconfiguration from:

- Memory safety violations, which depend on undefined behavior.
- Syntax or type errors, which violate language rules.
- External deployment misconfigurations that occur outside the compilation artifact.

The defining characteristic is that the program is locally correct under language semantics but globally insecure under a system-level invariant.

4.5 Trust-Boundary Leakage

We now instantiate this definition for trust-boundary leakage arising from object representation.

Let:

- o be an object with byte-level representation $\text{Bytes}(o)$.
- $\text{Pad}(o) \subseteq \text{Bytes}(o)$ denote padding bytes introduced for alignment [9, 10].
- $\text{Init}(o) \subseteq \text{Bytes}(o)$ denote the subset of bytes explicitly initialized by program logic.

A structural trust-boundary leakage occurs if:

$$\exists o, b \in B \text{ such that } o \text{ crosses } b \wedge \mathrm{Pad}(o) \not\subseteq \mathrm{Init}(o)$$

That is, an object crosses a trust boundary and contains padding bytes that were not explicitly initialized [11, 18, 19, 12, 13].

Importantly:

- The object layout conforms to ABI rules [10].
- No undefined behavior is required [9, 11, 14].
- The transfer operation is standards-compliant [9, 10].

Yet confidentiality invariant $I(P)$ is violated. Under the definition above, this condition constitutes a security semantic misconfiguration.

4.6 Layered Semantic Model

Figure 4.1 summarizes the relationship between language semantics, representation semantics, trust-boundary contexts, and security invariants.

Language semantics ensure correctness under the standard. Representation semantics determine how objects are laid out in memory. Trust-boundary contexts define when object representations become observable across domains. Security invariants constrain acceptable observable behavior. For the purposes of this thesis, we use the term *security semantic misconfiguration* to denote a condition in which a program is well-formed and does not rely on undefined behavior, yet nevertheless violates a security invariant because of representation or configuration semantics.

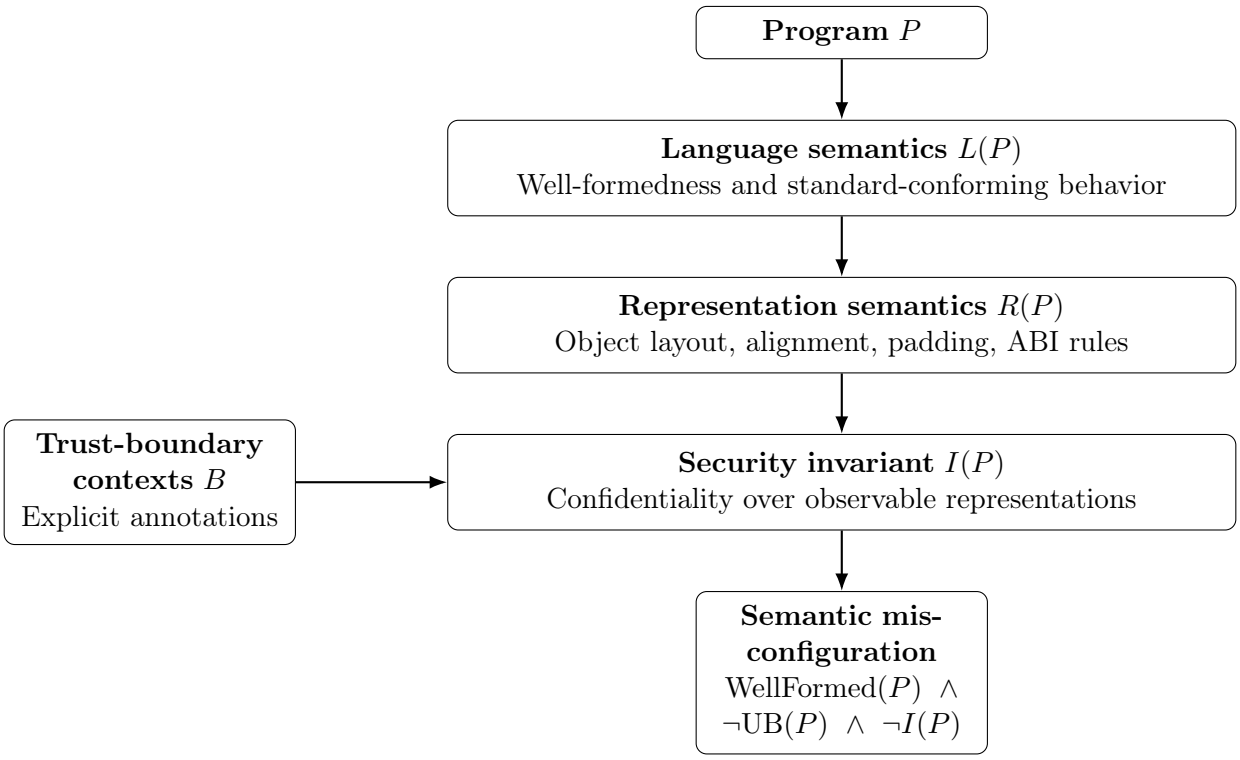

Figure 4.1: Layered abstraction relating language semantics, representation semantics, trust-boundary contexts, and security invariants.

Chapter 5

Taxonomy-Driven Integration

This thesis adopts a *taxonomy-driven* methodology: we first define a structured space of security misconfigurations that (i) occur in practice in C/C++ systems, (ii) reflect a mismatch between the developer's assumed guarantees and the program's real guarantees, and (iii) admit detection and explanation using compiler-visible semantics and integration into developer tooling [31, 1, 2, 4]. This approach is motivated by cryptographic misuse, a canonical domain where "correct-looking" code can silently violate security goals, and is instantiated in this thesis through a concrete case study on *trust-boundary information leakage* mediated by object-representation semantics [1, 2, 3, 4].

A key bridge is that cryptographic code is rarely isolated: secrets, derived values, and security-critical state cross *interfaces* (library APIs, module boundaries, RPC/IPC channels, enclave or privilege boundaries). When least-privilege boundaries are introduced (or even when libraries merely separate "trusted" from "untrusted" components), interface design becomes security-critical. The Compartment-Interface Vulnerability (CIV) literature systematizes this as a distinct vulnerability class: interfaces that used to be internal function calls become potential attacks between mutually distrusting compartments, and failures arise from insufficient vetting of cross-compartment data and control [8]. Within that CIV taxonomy, cross-compartment leakage includes leaks from *uninitialized data structures and compiler-added padding* (DL1/DL2) [8]. This observation directly motivates treating representation-level boundary leakage as a first-class misconfiguration family grounded in $R(P)$ and annotated boundary contexts B in this thesis' program model [11, 12, 13, 18, 19].

5.1 Taxonomy Requirements

In this thesis, the taxonomy is treated as a rule schema that supports [3, 4]:

1. **Semantic coherence:** each family corresponds to a meaningful security guarantee (confidentiality, integrity, authenticity, freshness, ordering).
2. **Operational anchors:** each family maps to recognizable code patterns (API calls, types, boundary transitions, configuration sites).
3. **Evidence structure:** each family specifies what counts as strong evidence vs weak evidence under compiler-visible analysis.
4. **Actionable remediation:** each rule family is paired with fix strategies that developers can apply without ambiguous interpretation.

These requirements reflect the thesis' goal of *detecting and explaining* misconfigurations inside a compiler-centric workflow (LLVM/Clang integrated into CodeChecker), under an explicit trust-boundary model [6, 7, 8, 4].

5.2 Top-Level Taxonomy

We organize security misconfigurations into three top-level families that capture the dominant failure modes relevant to this thesis' framing:

(A) Cryptographic Misuse Misconfigurations. Misconfigurations where the cryptographic *primitive, parameters, or usage protocol* fail to provide the intended security property (confidentiality, integrity, authenticity), despite syntactic correctness [1, 2, 3].

(B) Trust-Boundary and Interface Misconfigurations. Misconfigurations where data or control crosses a boundary with incompatible trust assumptions, and the interface is not designed or validated as a distrust boundary. This aligns with CIVs: interfaces become attack surfaces when compartments distrust each other [8].

(C) Data-Movement and Representation Misconfigurations. Misconfigurations where byte-level representation, layout, or copying semantics violate confidentiality or cause integrity-relevant nondeterminism.

This family includes representation leakage via padding, and structurally hazardous transfers that expose raw object representations across boundaries [11, 14, 15, 12, 13].

The three families frequently co-occur. Cryptographic guarantees are routinely mediated by interfaces and data movement, while boundary designs are implemented through byte-oriented operations whose behavior is governed by representation and layout. The taxonomy therefore classifies misconfigurations along two orthogonal axes: structural misconfigurations in program design (where security assumptions are encoded in module boundaries and data flows) and compiler-visible misconfigurations in representation and usage (where those assumptions are violated by concrete ABI and language-level semantics) [8, 11, 31].

5.2.1 Taxonomy Entry Schema

Each misconfiguration entry in the taxonomy is specified using a common schema so that (i) the security property is explicit, (ii) the operational manifestation is anchored to code patterns, and (iii) diagnostics and remediations are consistent [3, 4].

ID and Name. Stable identifier and short descriptive name.

Misconfiguration class. Cryptographic misuse / trust-boundary interface violation / data-movement representation misuse (entries may list secondary classes).

Violated security property. Confidentiality / Integrity-Authenticity / Availability.

Threat model and boundary preconditions. What trust partition is assumed; what must hold for the issue to be security-relevant (e.g., crosses a lower-trust domain, persistent sink, privilege boundary).

Operational pattern. Canonical API calls or syntactic/semantic patterns (e.g., `&obj` + `sizeof(obj)` passed to a byte-oriented sink; record passed/returned by value at a boundary).

Compiler-visible evidence. Evidence the checker can extract locally (AST shape, record layout/padding, dominance of whole-object initialization, argument alias structure, etc.).

Explanation obligation. What the diagnostic must explain in concrete terms (e.g., object representation vs field values; padding; boundary observation).

Remediation class. Zero-initialize whole object; field-wise serialization; canonical encoding; use AEAD/MAC; enforce call ordering; validate interface contract.

False-positive controls. Conditions that suppress or downgrade reporting (e.g., dominating `memset`, value-init, packed/no-padding, provably clean source).

5.2.2 Example Entry: Aggregate Authentication

ID and Name. DM-10 / "Digest or MAC over aggregate object representation."

Misconfiguration class. Data-movement representation misuse (primary); cryptographic misuse (secondary); trust-boundary sensitive when tags cross domains.

Violated security property. Integrity/authenticity (tag authenticates unintended bytes); confidentiality (if padding carries residual secrets and tags or transcripts are exposed).

Threat model and boundary preconditions. The program computes a tag/digest that is observable outside the trust domain (sent to peer, logged, stored, or checked across components).

Operational pattern. A byte-oriented cryptographic API consumes `(&obj, sizeof(obj))` where `obj` has record type.

Compiler-visible evidence. (i) `obj` is record type; (ii) record layout contains padding under the target ABI; (iii) `sizeof(obj)` matches consumed length; (iv) no dominating whole-object initialization is visible.

Explanation obligation. The diagnostic must state that the API consumes the in-memory image (object representation), so padding bytes

contribute to the authenticated input even though they are not semantic fields; consequently, the tag may depend on indeterminate or stale state.

Remediation class. Construct a canonical byte encoding from semantic fields and authenticate that buffer; alternatively, enforce whole-object initialization when authenticating the memory image is semantically intended and safe.

False-positive controls. Suppress/downgrade when there is dominating whole-object initialization (e.g., `T obj={0};` or `memset(&obj,0,sizeof obj)`), or when record layout contains no padding (explicit packing or naturally aligned fields).

5.2.3 Taxonomy Mapping

Table 5.1 summarizes how the taxonomy families map to compiler-visible anchors and what analysis power is required to implement them. This clarifies which misconfiguration classes are instantiated by the prototype checker and which remain as taxonomy-level categories for future extensions.

Table 5.1: Mapping from taxonomy families to compiler-visible anchors, analysis requirements, and current implementation status.

Taxonomy slice	**Compiler-visible anchors**	**Primary analysis requirements**	**Status**	**Typical diagnostics / fixes**
CM1–CM5 (Crypto misuse)	API identifiers; constant parameters; missing/unchecked return values; coarse call-sequence structure	API modeling; lightweight intraprocedural dataflow; optional limited interprocedural sequencing	Taxonomy only	Safer primitives/-modes; enforce verification; avoid constants; nonce/IV policy
TB-DL (Leakage; CIV)	Boundary annotations; by-value argument/return at boundary; directionality (entry/exit)	Boundary modeling; event extraction (E_B); explain cross-context observability	Instantiated	Warn on exposure; require explicit serialization; sanitize/zero-init

Taxonomy slice	Compiler-visible anchors	Primary analysis requirements	Status	Typical diagnostics / fixes
TB-DC (Corruption; CIV)	Boundary annotations; pointer/size/index fields crossing boundary; parsing idioms	Interprocedural contracts; taint/-dataflow; aliasing for attacker-controlled fields	Taxonomy only	Validate sizes/indices; defensive parsing; checked wrappers; defensive copying
TB-TV (Temporal; CIV)	Boundary annotations; sequencing assumptions (handshake/init/lock), TOCTOU patterns	Control-flow + concurrency reasoning; interprocedural sequencing; may require dynamic assumptions	Taxonomy only	Explicit state machines; synchronization; remove TOCTOU gaps
DM (Byte-defined sinks)	`&obj` + `sizeof(obj)`; known sinks (I/O, IPC, hashing, comparisons); record layout/padding	Sink recognition; layout/padding computation; local initialization evidence; optional alias reasoning	Partially inst.	Field-wise serialization; avoid hashing native structs; zero-init ABI/boundary structs
DM (Boundary-transfer by-value)	Pass/return-by-value of padded record at annotated boundary	Layout/padding + init heuristics; translation-unit structural matching	Implemented	Diagnose padding leakage; explain representation vs fields; suggest zero-init or serialization

5.3 Cryptographic Misuse Families

Cryptographic misconfigurations are most usefully described by the security property they disrupt: *confidentiality*, *integrity/authenticity*, or *availability*. In practice, the disruption is rarely caused by a syntactic error; it stems from violating usage constraints and best practices that the type system and compiler do not enforce [1, 2, 3, 4]. Accordingly, the taxonomy treats cryptographic misuse as a class of *security misconfiguration*: code that compiles and runs, yet undermines its intended protection because critical assumptions are left implicit [1, 2, 3].

5.3.1 CM: Cryptographic Misuse Families

CM1: Weak or Deprecated Primitive Selection. Examples include selecting obsolete hashes, ciphers, or key sizes that no longer meet the desired security margin. The observable signature is API selection and constant parameters (e.g., algorithm identifiers, bit-length arguments) [1, 2].

CM2: Missing Authentication / Integrity (Unauthenticated Encryption). Examples include using encryption without integrity protection when the application assumes authenticity (e.g., treating ciphertext as tamper-evident). Compiler-visible evidence includes use of encryption APIs without accompanying authentication APIs, or use of low-level cipher modes without MAC construction [1, 2, 3].

CM3: Nonce/IV Mismanagement. Examples include reuse of nonces/IVs, constant IVs, IV derived from low-entropy sources, or failure to randomize per message. Evidence often arises from constants, reuse of variables across calls, or suspicious initialization patterns [1, 2].

CM4: Key Management Misconfiguration. Examples include hard-coded keys, keys stored in non-secret memory regions, improper key derivation, or incorrect key lifetime handling. Evidence includes string/byte-array literals, improper key derivation calls, or key material crossing boundaries where it should remain confidential [1, 2, 3].

CM5: Protocol-Level Misuse and Ordering Violations. Examples include failure to verify certificates, skipping signature checks, ignoring return codes for verification, or violating required call order (init/update/final, handshake phases). Evidence includes missing calls, unchecked return values, and structurally inconsistent call sequences [3, 4].

Structural cryptographic misuse vs compiler-based misuse

Many cryptographic APIs consume or emit byte sequences whose meaning is defined outside the language's field-level semantics. This creates a recurring analysis question: whether the bytes supplied to security-critical interfaces are exactly the intended message/key/nonce material, or whether they accidentally include stale, indeterminate, or boundary-derived state. This question is compiler-visible because

it is expressed in terms of argument shapes (`&obj`, `sizeof(obj)`), record layout, and initialization structure, rather than programmer intent [2, 3, 4].

5.4 Trust-Boundary Misconfigurations

The CIV literature provides a systematic taxonomy of failures that arise when compartmentalizing software: interfaces must be designed under distrust, and failures lead to leakage, corruption, and temporal violations [8]. We incorporate these categories as a major axis of the thesis taxonomy because they characterize *what can go wrong at trust boundaries* independent of any specific cryptographic primitive.

5.4.1 TB: Trust-Boundary Misconfiguration Families

We adopt the CIV classes as a structured sub-taxonomy of trust-boundary failures:

TB-DL: Cross-Boundary Data Leakage.

- **TB-DL1 (Address/metadata exposure):** leaks of addresses or layout-derived information that undermine isolation defenses (e.g., ASLR).
- **TB-DL2 (Confidential data exposure):** leaks of secrets or sensitive state across a boundary.

CIV explicitly notes that DL1/DL2 leaks can stem from *uninitialized structures/-fields and compiler-added padding* [8].

TB-DC: Cross-Boundary Data Corruption.

- **TB-DC1:** dereference of corrupted pointers crossing the boundary.
- **TB-DC2:** use of corrupted indexing information (sizes, offsets, indices).
- **TB-DC3:** use of corrupted objects whose validity depends on API semantics.

These classes capture integrity violations and are central to why compartment interfaces must be designed as distrust boundaries [8].

TB-TV: Cross-Boundary Temporal Violations.

- **TB-TV1:** implicit ordering assumptions not enforced by the interface.
- **TB-TV2:** corrupted synchronization primitive usage across compartments.
- **TB-TV3:** time-of-check to time-of-use patterns on shared data.

Temporal classes connect directly to both cryptographic protocol ordering requirements (CM5) and interface design constraints under distrust [8].

5.5 Data-Movement Misconfigurations

A recurring theme across cryptographic misuse and CIVs is that interfaces frequently treat memory as a byte string. In C/C++, byte-oriented transfers can make the *entire object representation* observable, including padding bytes that may remain indeterminate or residual even when named fields are initialized. This is the representation-level hazard emphasized in the theoretical foundations and program model [9, 11, 14, 15, 12, 13].

5.5.1 DM: Representation Reads

We define a catalogue of boundary-transfer "representation read" rules. Each rule is presented with its sink pattern, why it constitutes a representation read, the primary failure mode, and the recommended remediation. These rules instantiate the general TB-DL family (leakage) at concrete, byte-oriented sinks and also connect back to cryptographic misuse when the sink is a cryptographic digest/MAC/signature API [11, 18, 19, 12, 13].

Table 5.2: Data-movement and representation misconfiguration rules (boundary-transfer representation reads).

Rule	Sink category	Common APIs / patterns	Primary failure mode	Preferred remediation
DM-01	File / persistent write	`write,pwrite,writev`; binary export paths	Persistent disclosure of stale padding / residual data	Whole-object zero-init before assignment; or field-wise serialization
DM-02	stdio binary output	`fwrite(&obj,sizeof obj,1,fp)`	Persistent disclosure via in-memory image write	Field-wise serialization; avoid native-struct persistence
DM-03	Network send (simple)	`send,sendto` with `&obj,sizeof obj`	Cross-domain disclosure (padding, residuals)	Build explicit wire format; do not send native structs
DM-04	Network send (scatter/-gather)	`sendmsg`+`iovec`, `writev`	Hidden leak through indirection; review-resistant	Copy fields into dedicated buffer; or zero-init object
DM-05	IPC / message queue / pipe	pipes/FIFOs, `mq_send`, project IPC wrappers	Cross-process disclosure; ABI-coupled behavior	Stable serialization format; avoid native-struct transfer
DM-06	Outbound buffer population	`memcpy/memmove` into TX/log/IPC buffer	Delayed disclosure through later sink	Serialize fields only; sanitize buffers at boundaries
DM-07	Serialization helpers	`append(buf,&obj,sizeof obj)`, `BIO_write`, wrappers	Leaks hidden inside abstraction layers	Treat helper as sink; enforce explicit encoding

Rule	Sink category	Common APIs / patterns	Primary failure mode	Preferred remediation
DM-08	Privilege boundary copy	`copy_to_user/copyout` (when in scope)	Privilege-crossing disclosure (high impact)	Mandatory zero-init; dedicated sanitized transfer structs
DM-09	ioctl / ABI marshalling	ioctl argument structs, `(get|set)sockopt` structs	Uninitialized padding leaks; ABI nondeterminism	Zero-init ABI structs prior to field assignment
DM-10	Hash / MAC / signature over bytes	digest/MAC update on `&obj,sizeof obj`	Nondeterministic digest; accidental secret inclusion	Canonical field-wise encoding prior to digest/-MAC/sign
DM-11	Raw aggregate comparison	`memcmp(&a,&b,sizeof a)`	False inequality; security decisions depend on padding	Compare semantic fields; canonicalize representation
DM-12	Binary logging / tracing	hexdump utilities, tracing probes over `&obj,sizeof obj`	Debug/telemetry leakage of residuals/secrets	Log fields explicitly; scrub/zero before logging
DM-13	DB / KV blob binds	`sqlite3_bind_blob`, KV blob writes	Long-lived leakage in storage/caches	Structured schema fields or explicit packed format
DM-14	Shared memory / ring buffer	memcpy into shared region, mmap ring protocols	Cross-thread/process leakage; replay nondeterminism	Protocol framing with explicit fields; zero-init
DM-15	FFI byte-array transfer	JNI/Python bytes, Rust-C FFI blobs	Leak into higher-level runtime / serialization	Define explicit FFI serialization layer

These DM rules formalize the operational meaning of the claim in Chapter 4 that "byte-level representations may become observable across domains with different confidentiality assumptions" by enumerating the byte-oriented sinks where representation semantics

$R(P)$ directly determine cross-domain observations [11, 18, 19, 12, 13]. They also connect cryptographic misuse (CM families) to trust boundaries: DM-10 captures a structurally hazardous misuse where developers authenticate or hash an in-memory aggregate representation rather than a canonical message, thereby allowing padding and residual bytes to influence the authenticated input [9, 11, 4].

5.6 Evidence Lattice

For the purposes of this thesis, diagnostic confidence is organized using an evidence lattice over compiler-visible facts extracted from the AST and layout model. A taxonomy that drives a static checker must specify not only *what* constitutes a misconfiguration class, but also *how strongly* a given program instance supports that classification under compiler-visible evidence. To make this explicit, we represent diagnostic confidence using a small evidence lattice. The lattice organizes observable facts extracted from the AST and layout metadata into discrete evidence levels and constrains diagnostic policy to be monotone with respect to evidence: adding evidence cannot reduce confidence, while removing evidence cannot increase it.

5.6.1 Evidence Domain and Partial Order

Let e be a candidate event instance (e.g., a boundary-transfer event or a byte-defined sink application) and let $\mathcal{F}(e)$ be the set of evidence facts derived by the checker at e. We partition facts into three conceptually distinct groups:

1. **Sink evidence** ($\mathcal{F}_S$): facts establishing that the event consumes bytes rather than semantic fields, e.g., a recognized sink API and corresponding argument shape.

2. **Representation evidence** ($\mathcal{F}_R$): facts establishing that the relevant type has a nontrivial object representation, e.g., record layout contains padding under the target ABI.

3. **Initialization evidence** ($\mathcal{F}_I$): facts establishing that the consumed representation may contain indeterminate or stale bytes, e.g., lack of dominating whole-object initialization along visible paths.

These groups reflect a necessary structure for the misconfiguration class considered here: the checker must establish a byte-defined observation mechanism (sink), the presence of representation artifacts (padding), and the plausibility of uninitialized representation state (initialization).

We define a partial order $\preceq$ over evidence sets by set inclusion:

$$\mathcal{F}(e)_1 \preceq \mathcal{F}(e)_2 \quad \text{iff} \quad \mathcal{F}(e)_1 \subseteq \mathcal{F}(e)_2.$$

A diagnostic policy is *monotone* if it never decreases confidence when additional facts are available:

$$\mathcal{F}(e)_1 \preceq \mathcal{F}(e)_2 \;\Rightarrow\; \text{Level}(\mathcal{F}(e)_1) \leq \text{Level}(\mathcal{F}(e)_2).$$

This monotonicity property ensures that improvements in analysis precision (e.g., more complete initialization reasoning) can only refine diagnostics upward rather than invalidating prior warnings.

5.6.2 Evidence Levels

For practical reporting, we collapse evidence sets into four discrete levels, $E0$–$E3$, each associated with a diagnostic policy. The levels are defined in terms of minimal fact patterns.

Level	Minimal evidence pattern	Diagnostic policy
$E0$	Candidate boundary context or sink is present, but no concrete byte-defined observation can be established.	No report.
$E1$	Byte-defined observation is established (**sink evidence**), but either (i) the consumed object is not a record type, or (ii) padding presence cannot be established under available layout metadata.	Suppressed by default; optionally report under "strict" mode.
$E2$	Byte-defined observation and padding presence are established (**sink + representation evidence**), but initialization cleanliness is uncertain under translation-unit visibility.	Warning with qualified language and remediation guidance.
$E3$	Sink, padding, and lack of dominating whole-object initialization are supported (**sink + representation + initialization evidence**), making exposure of indeterminate or stale representation bytes plausible.	Warning (high confidence) with explicit explanation tied to object representation and padding.

The intent is not that $E0$–$E3$ approximate a probability, but that they enforce a consistent thresholding discipline for what the checker may assert. In particular, $E2$ corresponds

to instances where the checker can show that the interface consumes representation bytes and that padding exists, but cannot fully discharge initialization due to limited flow/alias reasoning. $E3$ corresponds to instances where visible evidence indicates both padding and a likely absence of whole-object clearing. This calibrated reporting discipline is important for developer-facing tooling: it supports triage, makes the basis of a diagnostic explicit, and keeps emitted claims aligned with the strength of the available compiler-visible evidence.

5.6.3 Case-Study Instantiation

For the by-value boundary-transfer case study implemented in this thesis, sink evidence is obtained from (i) explicit boundary annotations and (ii) event extraction that identifies pass-by-value or return-by-value of record types at those boundaries. Representation evidence is obtained from record layout computation, including the existence and location of padding bytes under the target ABI. Initialization evidence is approximated using conservative dominance checks over common whole-object initialization idioms (e.g., `T obj={0};`), along with local syntactic cues indicating field-wise assignment patterns [6, 7]. Richer implementations can extend this evidence to explicit bytewise clearing idioms such as `memset(&obj,0,sizeof obj)`.

Under this instantiation, the checker reports $E3$ diagnostics when all three evidence groups are satisfied, and reports $E2$ diagnostics when initialization evidence is incomplete but sink and representation evidence are strong. $E1$ and $E0$ are not typically reported in the default configuration.

5.6.4 Evidence Facts

The following conditions are treated as evidence within the lattice.

Sink evidence ($\mathcal{F}_S$).

- Boundary-event shape matches a by-value transfer at an annotated boundary, or a recognized byte-defined sink consumes `(&obj, sizeof(obj))`.
- The consumed source expression denotes a concrete aggregate object (not an opaque pointer to unknown storage).
- The size argument is exactly `sizeof(obj)` or `sizeof(struct T)`.

Representation evidence ($\mathcal{F}_R$).

- The transferred/consumed type is a record type whose computed layout contains padding bytes under the target ABI.
- Nested aggregates increase padding surface (supporting explanation and remediation priority).

Initialization evidence ($\mathcal{F}_I$).

- The object has automatic storage duration with no dominating whole-object initialization or clearing operation visible in the translation unit.
- The object is field-assigned prior to the sink (a common pattern in which padding remains untouched).

Confidence-reducing facts. Evidence of dominating whole-object initialization decreases confidence and may suppress reporting or reduce diagnostic severity. Such facts are incorporated by mapping an instance to a lower evidence level (e.g., $E3 \rightarrow E2$ or $E2 \rightarrow E1$) depending on what remains established.

- A dominating whole-object initialization is observed via a recognized policy-approved pattern (e.g., `T obj={0};`); richer implementations can additionally recognize explicit bytewise clearing idioms such as `memset(&obj,0,sizeof obj)`.
- The object is provably value-initialized such that the full representation is cleared.
- The object is copied from a known-clean, fully initialized representation.
- The type is explicitly packed and has no padding (though it may still have other uninitialized-field issues).

This evidence discipline is designed to be implementable without whole-program flow tracking, consistent with the thesis' emphasis on translation-unit structural analysis and conservative initialization heuristics.

Chapter 6

Modeling Framework

This chapter refines the semantic objects introduced in Chapter 4 into a concrete, *checker-oriented model* whose purpose is to generate boundary-transfer candidates and to decide, from compiler-visible evidence, whether a candidate plausibly violates the confidentiality invariant $I(P)$. The result is an operational specification for the prototype: it defines (i) which event kinds constitute E_B in the case study, (ii) what byte-level observation means for those events, (iii) how padding is derived from record layout, and (iv) how initialization is approximated in a way that aligns with the evidence lattice of Chapter 5.

6.1 Model Interface

Inputs. The model assumes the following inputs are available to a compiler-based analysis:

1. A program P that is well-formed and does not invoke undefined behavior.
2. An explicitly specified set of trust-boundary interfaces B (via source-level annotations).
3. Target-dependent layout metadata for record types under the compilation ABI, exposed through Clang's semantic model [6, 10].

Outputs. Given these inputs, the model defines:

1. A set of boundary-transfer events E_B for the case study.
2. For each event instance $e \in E_B$, an *observation* $\mathrm{Obs}(e)$ defined over bytes.
3. A leakage predicate $\mathsf{Leak}(e)$ that approximates $\neg I(P)$ at e.

4. An evidence level $\mathrm{Level}(e) \in \{E0, E1, E2, E3\}$ as in Chapter 5, supporting systematic reporting policy.

Scope. This chapter instantiates E_B to a restricted but high-evidence class of events: record objects transferred *by value* across annotated boundaries via the call/return ABI [10]. Pointer-mediated transfers and sink-based byte operations (write/send/hash) are treated as staged extensions (cf. Table 5.1), and are not required to state the core model.

6.2 Boundary Transfer Events

Let $B \subseteq \mathcal{D} \times \mathcal{I} \times \mathcal{D}$ denote the set of directed boundary interfaces from Chapter 4. For the implemented case study, the boundary-transfer event set E_B is defined as:

$$\begin{aligned} E_B \triangleq\ & \{\, \mathsf{call}(i, o) \mid (d_s, i, d_t) \in B \wedge o \text{ is a record argument passed by value at } i \,\} \\ & \cup\ \{\, \mathsf{ret}(i, o) \mid (d_s, i, d_t) \in B \wedge o \text{ is a record value returned by value at } i \,\}. \end{aligned}$$

Why by-value events are privileged. A by-value call/return exposes the record's ABI-level transferred representation at the interface boundary [10]. Unlike pointer or reference passing, which requires alias reasoning to determine what bytes are later observed, a by-value transfer provides a direct structural anchor for byte-level observation at the interface. This choice aligns with the thesis goal of evaluating what can be achieved with compiler-visible structural semantics without whole-program flow tracking [5, 25].

Minimal operational semantics. For the purposes of this case study, we model each event $e \in E_B$ as inducing an *interface observation*:

$$\mathrm{Obs}(e) \triangleq \mathrm{Bytes}(o) \in \{0, 1\}^{8 \cdot \mathrm{sizeof}(T)},$$

i.e., the observed artifact is the full object representation of the transferred record object o of static type T at the moment it is transferred according to the call/return ABI [9, 10]. This definition is consistent with the cross-domain observation notion introduced in Chapter 4: if two executions produce different $\mathrm{Bytes}(o)$ at the boundary, then the target domain can (in principle) distinguish them at the interface [11, 15].

6.3 Object Representation and Padding

Let o be a concrete record object of static record type T.

Byte-level representation. Bytes(o) denotes the contiguous byte sequence occupied by o in memory [9, 16]. Fields(T) denotes the declared fields of T.

Padding bytes. Pad(o) $\subseteq$ Bytes(o) denotes the subset of bytes in Bytes(o) that are not covered by the value representations of any declared fields. These bytes arise from ABI- and compiler-determined layout constraints (alignment and tail padding) and are present even when all declared fields are assigned [9, 16, 10].

Layout-derived characterization. Let size(T) be the size in bytes of T and let each field $f_i \in \mathrm{Fields}(T)$ have a byte offset $\mathrm{off}(f_i)$ and width $\mathrm{width}(f_i)$. Define the covered intervals:

$$\mathrm{Cover}(T) \triangleq \bigcup_{f_i \in \mathrm{Fields}(T)} [\,\mathrm{off}(f_i),\ \mathrm{off}(f_i) + \mathrm{width}(f_i)\,),$$

and the full storage interval $[0, \mathrm{size}(T))$. Then Pad(o) corresponds to the complement:

$$\mathrm{Pad}(T) \triangleq [0, \mathrm{size}(T)) \setminus \mathrm{Cover}(T), \qquad \mathrm{Pad}(o) \equiv \mathrm{Pad}(T) \text{ instantiated on } \mathrm{Bytes}(o).$$

We write $\mathsf{HasPadding}(T)$ iff $\mathrm{Pad}(T) \neq \varnothing$.

Compiler-visible availability. All quantities above are available through Clang's record layout metadata (field offsets, widths, and total size), making $\mathsf{HasPadding}(T)$ a purely structural, compiler-visible predicate [6].

6.4 Initialization Approximation

The confidentiality invariant $I(P)$ (Chapter 4) is violated at a boundary event when the cross-domain observation Obs(e) depends on bytes outside the intended logical state. For record objects, this reduces to whether padding bytes may be uninitialized at the event [9, 11, 14].

Let Init(o) $\subseteq$ Bytes(o) denote the subset of bytes whose values are established by program actions prior to the event. Determining Init(o) precisely requires flow- sensitive reasoning and alias tracking, which is out of scope for the case study [5, 25]. Instead, we define an approximation $\widehat{\mathrm{Init}}(o)$ designed to be implementable from translation-unit structural evidence.

Initialization classes. We distinguish three structural initialization regimes:

Whole-object initialization. Evidence that the full object representation is initialized (e.g., an explicit bytewise clear such as `memset(&o,0,sizeof o)`, or a source-visible

zero-initialization idiom for the entire record object) [18, 19, 12, 13]. In this case, $\widehat{\mathrm{Init}}(o) \triangleq \mathrm{Bytes}(o)$.

Field-wise initialization. Evidence that declared fields are assigned but no whole-object initialization is visible. In this case, $\widehat{\mathrm{Init}}(o) \triangleq \mathrm{Cover}(T)$.

Unknown initialization. No reliable evidence is visible. In this case, $\widehat{\mathrm{Init}}(o) \triangleq \varnothing$.

Approximation principle. Padding is treated as initialized only under whole-object initialization evidence. This is conservative with respect to confidentiality: it may over-warn when padding is cleared indirectly, but it avoids incorrectly asserting safety when only field assignments are visible.

6.5 Leakage Predicate

For a boundary-transfer event $e \in E_B$ involving a record object o of type T, we define the event-level leakage predicate:

$$\mathsf{Leak}(e) \triangleq \mathsf{HasPadding}(T) \ \wedge \ \big(\mathrm{Pad}(T) \not\subseteq \widehat{\mathrm{Init}}(o)\big).$$

Operationally, detection for an event instance proceeds by the following pipeline:

1. Identify an event instance $e \in E_B$ (call-by-value or return-by-value at an annotated boundary).
2. Determine the static record type T involved in the by-value transfer.
3. Compute $\mathsf{HasPadding}(T)$ from record layout.
4. Approximate $\widehat{\mathrm{Init}}(o)$ using local structural initialization evidence.
5. Report if $\mathsf{Leak}(e)$ holds, with evidence level as described below.

Alignment with the evidence lattice. The model interfaces directly with the evidence lattice in Chapter 5:

- $\mathcal{F}_S$ (sink evidence) corresponds to establishing $e \in E_B$ under explicit boundary annotation.
- $\mathcal{F}_R$ (representation evidence) corresponds to $\mathsf{HasPadding}(T)$.
- $\mathcal{F}_I$ (initialization evidence) corresponds to structural proof of whole-object initialization (or its absence) used to determine $\widehat{\mathrm{Init}}(o)$.

Thus, $E3$ corresponds to strong evidence for $e \in E_B$ and $\mathsf{HasPadding}(T)$ together with visible absence of whole-object clearing, while $E2$ corresponds to strong sink+representation evidence with incomplete initialization evidence.

6.6 Worked Examples

The following minimal examples illustrate how the model is applied. They are intentionally small so that B, E_B, $\mathsf{HasPadding}$, and $\widehat{\mathrm{Init}}$ can be instantiated explicitly.

6.6.1 Example 1: Field-wise Initialization

Consider a record type T with likely padding under a typical ABI [10]:

```
struct Msg { uint8_t tag; uint64_t x; uint32_t n; };
TRUST_BOUNDARY void send_msg(struct Msg m);
```

Let B contain $(d_s, \texttt{send_msg}, d_t)$ and consider:

```
struct Msg m;
m.tag = 1; m.x = 42; m.n = 7;
send_msg(m);
```

Here $\mathsf{call}(\texttt{send_msg}, m) \in E_B$ and $\mathrm{Obs}(e) = \mathrm{Bytes}(m)$ by definition. Under common ABIs, $\mathsf{HasPadding}(T)$ holds due to alignment between `tag` and `x` and potential tail padding [10]. Field-wise assignments establish $\widehat{\mathrm{Init}}(m) = \mathrm{Cover}(T)$, so $\mathrm{Pad}(T) \not\subseteq \widehat{\mathrm{Init}}(m)$ and $\mathsf{Leak}(e)$ holds. The model therefore predicts a warning: the boundary observation may include indeterminate padding bytes [11, 14].

6.6.2 Example 2: Whole-Object Initialization

Using the same type and boundary:

```
struct Msg m = {0};      // whole-object zero initialization
m.tag = 1; m.x = 42; m.n = 7;
send_msg(m);
```

The event is again in E_B and $\mathsf{HasPadding}(T)$ still holds. However, the initialization regime is whole-object initialization, so $\widehat{\mathrm{Init}}(m) = \mathrm{Bytes}(m)$ and $\mathrm{Pad}(T) \subseteq \widehat{\mathrm{Init}}(m)$. Hence $\mathsf{Leak}(e)$ is false and no warning is expected under the model.

6.7 Modeling Assumptions

6.7.1 Assumptions

The model and its prototype instantiation assume:

- **Explicit boundaries.** Membership in B is defined by source annotations; no boundary inference is attempted.
- **Translation-unit visibility.** Record definitions are available in the analyzed translation unit for layout computation.
- **Structural initialization evidence.** Only syntactically visible whole-object initialization patterns are used to establish $\widehat{\mathrm{Init}}(o)$.
- **No alias tracking.** Pointer-based transfers and indirect initialization through aliases are out of scope.

These assumptions are consistent with the thesis objective: to measure the expressive power of compiler-visible structural reasoning for approximating a restricted confidentiality invariant.

6.7.2 Staged Extensions

The model is intentionally parameterized so that additional event kinds can be incorporated by changing the definition of E_B and the observation function Obs while preserving the core idea that confidentiality is constrained over *representation bytes*. Two natural extensions are:

1. **Byte-defined sinks.** Extend E_B to include events where a byte-oriented sink consumes $(\&o, \mathrm{sizeof}(T))$ (e.g., write/send/hash). In this case, $\mathrm{Obs}(e)$ is the byte string consumed by the sink [12, 13].
2. **Buffer-mediated transfers.** Extend E_B to include population of outbound buffers followed by a sink. This requires dataflow to relate buffer writes to the boundary observation [5, 25].

The case study implemented in this thesis isolates the highest-evidence subset (by-value ABI transfers), allowing evaluation without conflating boundary leakage with alias tracking or interprocedural flow.

Chapter 7

Implementation as a CodeChecker Extension

This chapter describes the implementation of the analysis as a custom Clang-Tidy module executed through CodeChecker [6, 7]. The prototype operationalizes the modeling framework of Chapter 6 by (i) instantiating the boundary set B from explicit source annotations, (ii) extracting boundary-transfer events E_B (call-by-value and return-by-value of record objects), (iii) computing layout-derived padding predicates $\mathsf{HasPadding}(T)$ from compiler metadata, (iv) approximating whole-object initialization $\widehat{\mathrm{Init}}(o)$ using conservative structural heuristics, and (v) emitting diagnostics when the event-level predicate $\mathsf{Leak}(e)$ holds.

7.1 Integration Architecture

CodeChecker orchestrates static analysis over a project using a compilation database (`compile_commands.json`) [7]. In this work, CodeChecker is configured to execute Clang-Tidy on each translation unit in that database [6, 7]. The checker is implemented as an out-of-tree Clang-Tidy module distributed as a shared library, loaded at analysis time and enabled using standard Clang-Tidy configuration mechanisms [6].

This architecture provides three practical advantages: (1) it preserves access to compiler-visible semantic information (AST nodes, types, and layout), (2) it supports reproducible evaluation on real-world projects by reusing compilation databases, and (3) it produces diagnostics in a standardized format compatible with CodeChecker reporting and suppression workflows [6, 7].

7.2 Boundary Annotations

Trust boundaries are modeled via explicit source-level annotations. A boundary interface is declared using the Clang/GCC-style `annotate` attribute [6]:

```
#define TRUST_BOUNDARY __attribute__((annotate("trust_boundary")))
TRUST_BOUNDARY
int boundary_send(struct Msg m);
```

Within Clang, the annotation string is preserved as an `AnnotateAttr` attached to the corresponding `FunctionDecl` [6]. The analysis instantiates B as the set of annotated functions whose annotation payload matches the designated boundary marker. This design makes membership in B deterministic and compiler-visible, and isolates representation-level reasoning from boundary inference (treated as future work).

7.3 Event Extraction

The implementation instantiates the event set E_B from Chapter 6 by extracting two event kinds:

Call-by-value events. At a call site, the checker identifies a `CallExpr` whose callee resolves to a function in B. For each argument, it inspects the argument's static type and records an event instance $\mathsf{call}(i, o)$ when a record type T is passed by value.

Return-by-value events. Inside a boundary function definition, the checker identifies `ReturnStmt` nodes and records an event instance $\mathsf{ret}(i, o)$ when a record value of static type T is returned by value.

These event classes are chosen because they correspond to transfers that expose the record's ABI-level representation at the interface boundary [10]. Pointer or reference passing is intentionally excluded from the core implementation because it does not directly identify the observed bytes and typically requires alias analysis to interpret precisely [5, 25].

7.4 Type Selection

For each extracted event $e \in E_B$, the checker determines the static record type T of the transferred value. The analysis restricts attention to record types for which complete layout metadata is available to the current translation unit:

- T is a record type (`struct` or eligible `class` object).
- The record definition is available such that Clang can compute an `ASTRecordLayout` [6].

These restrictions ensure that the predicate $\mathsf{HasPadding}(T)$ can be computed deterministically from compiler metadata.

7.5 Padding Computation

The leakage predicate depends on whether the transferred type contains padding bytes in its object representation [9, 16, 10]. The checker computes $\mathsf{HasPadding}(T)$ from Clang record layout metadata [6].

Let $\mathrm{size}(T)$ be the total byte size of T. For each declared field f_i, Clang provides:

- a field byte offset $\mathrm{off}(f_i)$ (via `ASTRecordLayout::getFieldOffset`),
- and a field width $\mathrm{width}(f_i)$ (via the field type size in the current `ASTContext`) [6].

Padding existence is determined by detecting uncovered gaps in $[0, \mathrm{size}(T))$:

1. Sort fields by offset and compute each covered interval $[\,\mathrm{off}(f_i),\ \mathrm{off}(f_i) + \mathrm{width}(f_i)\,)$.
2. If any consecutive intervals exhibit a gap, then padding exists.
3. If the last covered interval ends before $\mathrm{size}(T)$, then tail padding exists.

Only events involving types with $\mathsf{HasPadding}(T)$ are candidates for leakage diagnostics.

7.6 Initialization Heuristics

Determining whether padding bytes are initialized requires flow-sensitive reasoning. Consistent with Chapter 6, the implementation uses conservative, translation-unit structural heuristics to approximate whole-object initialization $\widehat{\mathrm{Init}}(o)$ [5, 25].

Whole-object initialization evidence. The checker treats an object as fully initialized when it observes a recognized syntactic pattern that implies whole-object initialization, most notably source-visible zero-initialization idioms such as `T obj={0};` [18, 19, 12, 13]. The implementation is designed so that explicit bytewise clearing idioms such as `memset(&obj,0,sizeof obj)` can be incorporated into the same category, but the current prototype does not yet recognize every such pattern uniformly; this implementation gap is discussed in Chapter 8 and §9.1.

Field-wise initialization evidence. If only member assignments are visible without whole-object clearing evidence, the checker assumes field-wise initialization and treats padding as potentially uninitialized [11, 14].

This approximation is conservative with respect to confidentiality: it may over-report when objects are initialized indirectly, but it avoids asserting safety solely from field assignments.

7.7 Leak Evaluation

For each extracted event instance $e \in E_B$ with record type T, the implementation evaluates the event-level predicate of Chapter 6:

$$\mathsf{Leak}(e) \triangleq \mathsf{HasPadding}(T) \ \wedge \ (\mathrm{Pad}(T) \not\subseteq \widehat{\mathrm{Init}}(o)).$$

Diagnostics are assigned an evidence level consistent with the lattice of Chapter 5:

- **E2 (bounded confidence):** sink and representation evidence are strong ($e \in E_B$ and $\mathsf{HasPadding}(T)$), but whole-object initialization cannot be established or refuted reliably under translation-unit visibility.
- **E3 (high confidence):** sink and representation evidence are strong and visible structural evidence indicates absence of whole-object clearing, making exposure of padding plausible under the model.

By default, the prototype reports E2 and E3 findings and suppresses weaker categories.

7.8 Diagnostics and Reporting

When $\mathsf{Leak}(e)$ holds, the checker emits a diagnostic at the boundary transfer site (call expression or return statement). Each diagnostic includes:

- the transfer location and boundary callee context,
- the record type T and its size $\mathrm{size}(T)$,
- an explanation that the by-value boundary mechanism observes the object representation, and that T contains padding bytes that may remain indeterminate under field-wise initialization [10, 11, 14],
- the assigned evidence level (E2 or E3),

- a remediation suggestion: (i) whole-object initialization when semantically appropriate, or (ii) field-wise serialization into an explicit boundary buffer when transferring native struct bytes is conceptually incorrect for the interface [18, 19, 12, 13].

CodeChecker aggregates diagnostics across translation units into a single report, enabling triage and evaluation [7]. The explicit evidence levels are therefore not merely descriptive: they calibrate what the checker claims at each site and make the basis for that claim visible to the developer reviewing the report.

7.9 Implementation Scope

The prototype is intentionally limited to translation-unit structural reasoning: it does not infer trust boundaries automatically, track initialization interprocedurally, or perform alias analysis for pointer-based transfers. These limitations match the thesis objective: evaluating the expressive power of compiler-visible structural semantics for approximating representation-level boundary leakage under explicit trust-boundary assumptions [5, 25, 8].

Chapter 8

Evaluation

This chapter presents the evaluation protocol and empirical results for the prototype detector described in Chapters 6 and 7. The synthetic benchmark suite provides controlled correctness validation, while the real-world case studies characterize the event surface induced by the modeled boundary-transfer events in selected C libraries under explicit boundary annotation, and whether the evidence lattice assigns classifications consistent with the visible structural evidence.

The objective is characterization, not vulnerability mining. We measure where modeled boundary-transfer events arise under explicit boundary annotation, how often those events involve padded record types, how frequently conservative initialization evidence leads to suppression, and what observed patterns reveal about the interaction between API design conventions and the modeled leakage class.

The remainder of this chapter is organized as follows. Section 8.1 states the evaluation research questions. Section 8.2 describes the selected projects and dataset criteria. Section 8.3 details the annotation protocol. Section 8.4 defines the metrics and data collection procedure. Section 8.5 validates the implementation on synthetic benchmarks. Section 8.6 presents empirical results across four real-world projects. Section 8.7 reports performance overhead. Section 8.8 discusses threats to validity, and Section 8.9 summarizes the chapter's findings.

8.1 Research Questions

The evaluation addresses the following questions:

1. **Event surface.** How frequently do annotated boundary-transfer events E_B arise in C libraries when representative public interfaces are annotated?

2. **Representation surface.** When E_B events arise, how often do the transferred record types satisfy $\mathsf{HasPadding}(T)$?

3. **Evidence sensitivity.** How often does whole-object initialization evidence suppress findings or reduce confidence under the evidence lattice (§5.6)?

4. **Design implications.** What do observed patterns suggest about API design practices (by-value aggregates versus pointer and handle patterns) and their interaction with the modeled leakage class?

These questions are addressed systematically in §8.6.3.

8.2 Dataset Selection

8.2.1 Selection Criteria

Projects are selected to support reproducible analysis under CodeChecker and to represent diverse C programming contexts and API design philosophies [7]. The selection criteria are:

1. Primarily written in C (C-heavy codebases), Free and Open Source Software.

2. Builds cleanly with Clang under CMake and supports compilation database generation (`compile_commands.json`) [6].

3. Exposes a stable public API surface with identifiable domain-transition points suitable for explicit boundary annotation.

4. Representative of systems-oriented software where aggregate types and ABI constraints are relevant design concerns.

5. Covers a range of API design philosophies with respect to value semantics versus pointer-based data transfer, to characterize how design choices interact with the modeled leakage class.

The final criterion is deliberate: a dataset composed exclusively of pointer-based libraries would confirm only the negative case. By including libraries that employ value-semantic APIs, the evaluation can characterize both the event surface of the modeled leakage class and the checker's diagnostic behavior when the structural preconditions are met.

Project	Domain	API design rationale
zlib	Data compression	Mature opaque-handle API; expected $\|E_B\| = 0$
libuv	Async I/O	Predominantly pointer-based; one by-value return
raylib	Game graphics	Value-semantic philosophy; by-value returns throughout
Chipmunk2D	2D physics engine	Field-wise struct construction; no defensive zero-init

Table 8.1: Evaluated projects, application domain, and API design rationale for inclusion.

8.2.2 Selected Projects

The four evaluated real-world projects are **zlib**, **libuv**, **raylib**, and **Chipmunk2D**. Table 8.1 summarizes each project and its relevance to the evaluation.

zlib and **libuv** represent the established pattern in mature C systems libraries: opaque or pointer-based APIs that minimize aggregate transfer at external boundaries. These projects serve as the null-case baseline.

raylib is a widely-used game programming library that explicitly prioritizes value semantics. Functions such as `GetRayCollisionSphere`, `GetRayCollisionBox`, and related entry points return `RayCollision`, a struct with confirmed ABI padding, by value. This design is intentional and well-documented, making raylib a representative case of the API pattern the checker targets.

Chipmunk2D is a mature 2D rigid-body physics engine. Inspection of its source reveals that `cpArbiterGetContactPointSet` and `cpShapesCollide` return `cpContactPointSet` by value using field-wise initialization with no prior whole-object zero-initialization. The 4-byte padding gap between the `int count` field and the `cpVect normal` field (which requires 8-byte alignment) is never cleared, satisfying the structural leakage predicate. This makes Chipmunk2D the primary positive-case study in the real-world evaluation.

8.3 Boundary Annotation Protocol

Because boundary inference is out of scope for this prototype, boundaries are provided explicitly via the `TRUST_BOUNDARY` attribute described in Chapter 7. For each project, between three and five representative public API entry points are annotated as trust-boundary interfaces. Across the four evaluated projects this yields 18 annotated boundary interfaces in total. Not all annotated interfaces produce boundary-transfer events under the case-study definition; the event-producing subset is reported as $|B|$ in the empirical tables.

Boundary selection rule. Functions are selected to approximate realistic domain transitions:

- they are externally visible entry points on the public API surface,
- they accept or return structured aggregate types where possible,
- they represent a conceptual transition between library-internal state and external callers.

Where a library's public API does not expose functions accepting or returning aggregates by value, functions closest to the external interface are selected. The absence of by-value aggregate transfers is itself a measurable and meaningful outcome.

Recorded boundary set. For each project the following are recorded:

- $|B|$: number of annotated boundary functions that produce at least one boundary-transfer event,
- the annotated function identifiers (file and signature) to support reproducibility.

8.4 Measurement Plan

This section defines the concrete measurements reported in the empirical results. All measurements are derived from CodeChecker output, checker-side event logging, and the recorded boundary set [7].

8.4.1 Metrics

For each project, the following quantities are recorded:

$|B|$ Number of annotated boundary interfaces that produce at least one boundary-transfer event.

$|E_B|$ Number of boundary-transfer events extracted under the case-study definition: call-by-value and return-by-value transfers of record types at annotated boundary functions.

$N_{\mathbf{rec}}$ Number of *distinct* record types T participating in E_B events.

$N_{\mathbf{pad}}$ Number of participating record types satisfying $\mathsf{HasPadding}(T)$.

N_{E2}, N_{E3} Number of emitted diagnostics at evidence level E2 and E3 respectively.

$N_{\mathbf{sup}}$ Number of candidate events suppressed due to whole-object initialization evidence: candidates for which $\mathsf{HasPadding}(T)$ holds but which are mapped to a lower evidence level under the default policy.

Derived rates. In addition to raw counts, the following normalized rates are reported:

$$r_{\text{event}} = \frac{|E_B|}{|B|}, \quad r_{\text{pad}} = \frac{N_{\text{pad}}}{\max(1, N_{\text{rec}})}, \quad r_{\text{diag}} = \frac{N_{E2} + N_{E3}}{\max(1, |E_B|)}.$$

These rates normalize across projects and support comparison of API design choices independently of project scale.

8.4.2 Data Collection

All measurements are derived from three sources: (i) the recorded annotation set instantiating B, (ii) structured event records emitted by the checker to a CSV log file at analysis time, and (iii) CodeChecker-aggregated diagnostics across translation units [7].

Event records. The checker emits a structured event record for every extracted boundary-transfer instance, regardless of whether a warning is emitted. Each record captures the project and translation unit, event kind (`call` vs `ret`), boundary function identifier, static record type name and size, $\mathsf{HasPadding}(T)$ result, padding byte count, initialization class, evidence level, and diagnostic emission flag. Aggregating these records yields $|E_B|$, N_{rec}, N_{pad}, and N_{sup} directly.

Diagnostic counts. N_{E2} and N_{E3} are derived from event records filtered on `diag_emitted=true` and the corresponding evidence level field, cross-validated against CodeChecker-aggregated diagnostic reports.

Runtime overhead. For each project, wall-clock analysis time is recorded from CodeChecker's run log. Overhead is computed as:

$$\text{Overhead}(\%) \triangleq 100 \cdot \frac{t_{\text{with}} - t_{\text{base}}}{\max(1, t_{\text{base}})}.$$

8.5 Synthetic Benchmark Validation

The synthetic benchmark suite (Appendix A) validates that the implementation operationalizes the predicate of Chapter 6 under controlled conditions: padding versus no-

Project	**TUs**	$\|B\|$	$\|E_B\|$	N_{rec}	N_{pad}	N_{E2}	N_{E3}
synthetic	–	8	8	4	2	5	0
zlib	37	0	0	0	0	0	0
libuv	459	1	2	1	0	0	0
raylib	29	5	20	3	1	0	0
Chipmunk2D	92	3	6	2	1	4	0

Table 8.2: Empirical summary: translation units analyzed, annotated boundary interfaces, extracted events, participating record types, padding presence, and emitted diagnostics by evidence level.

padding, pass-by-value versus return-by-value, whole-object initialization versus field-wise initialization, and nested aggregates.

Validation criterion. For each benchmark case, the observed diagnostic outcome must match the expected result in Table A.1. The suite therefore functions as a regression harness: changes to event extraction, layout analysis, or initialization heuristics are accepted only if benchmark outcomes remain consistent.

Observed synthetic results. Across the eight synthetic boundary interfaces, the detector extracted eight boundary events involving four record types. Two of these record types contained ABI padding, resulting in five emitted E2 diagnostics. Four of these correspond to the expected positive benchmark cases, while the fifth (case 02) is the known `memset`-pattern false positive discussed below. One additional candidate was suppressed due to syntactically evident whole-object initialization (`{0}` pattern), yielding $N_{\text{sup}} = 1$. The resulting synthetic rates were $r_{\text{pad}} = 0.500$ and $r_{\text{diag}} = 0.625$.

These results confirm that the operational predicate implemented by the checker faithfully reflects the structural leakage invariant defined in Chapter 6. Diagnostics are emitted when padding exists and initialization evidence is absent, and suppressed when the invariant is syntactically satisfied.

8.6 Empirical Results

Tables 8.2 and 8.3 present the boundary interfaces, extracted events, record types, padding presence, diagnostics, suppressions, and derived rates across the synthetic benchmark suite and the four evaluated libraries.

Project	N_{sup}	r_{pad}	r_{diag}	r_{event}
synthetic	1	0.500	0.625	1.000
zlib	0	0.000	0.000	–
libuv	0	0.000	0.000	2.000
raylib	8	0.333	0.000	4.000
Chipmunk2D	0	0.500	0.667	2.000

Table 8.3: Suppression and rate metrics: whole-object initialization suppression count, and normalized padding, diagnostic, and event rates.

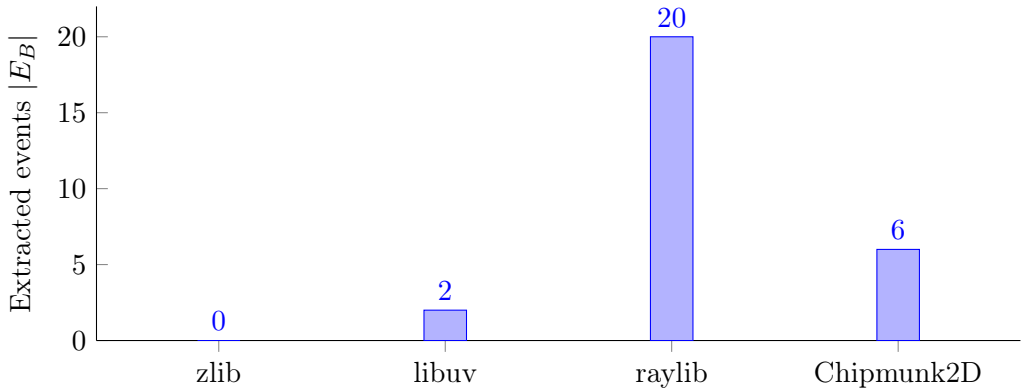

Figure 8.1: Distribution of extracted boundary-transfer events across the four evaluated real-world projects. The observed event surface varies strongly with API design philosophy: pointer-based libraries expose little or no event surface, whereas value-semantic libraries expose substantially more by-value aggregate transfers.

8.6.1 Per-Project Analysis

The four real-world projects produce four qualitatively distinct outcomes that together characterize the checker's behavior across the range of the evidence lattice.

zlib: pointer-based API, zero event surface. Five entry points were annotated (`deflate`, `deflateEnd`, `inflate`, `inflateEnd`, `deflateSetHeader`), all of which accept only pointer parameters at the external interface: specifically, `z_streamp`, a pointer to an opaque stream structure. The checker correctly extracts zero boundary-transfer events across all 37 translation units, producing no diagnostics. This outcome is reported as $|B| = 0$ in Table 8.2, reflecting the absence of by-value aggregate transfers at these interfaces rather than an absence of annotation. The result is consistent with the design philosophy of mature compression libraries: externally visible state is mediated through pointers or opaque handles, and the object representation never crosses the external interface by value.

libuv: single by-value return, no padding on target platform. Among the five annotated entry points, only `uv_buf_init` returns a record type by value. The libuv

Project	Type	Padding	Outcome
libuv	`uv_buf_t`	No	No warning
raylib	`RayCollision`	Yes	Suppressed by whole-object init
Chipmunk2D	`cpContactPointSet`	Yes	E2 diagnostic emitted

Table 8.4: Representative project-level outcomes illustrating the main paths of the evidence lattice: non-padded type, padded-but-suppressed type, and padded type with emitted diagnostic.

documentation explicitly acknowledges that the member ordering of `uv_buf_t` is platform-dependent; on Windows the struct layout introduces a 4-byte alignment gap between the `ULONG len` and `CHAR* base` members, but on Linux x86-64 both members are 8-byte aligned and no padding is inserted. The checker correctly extracts two E_B events (one per translation unit containing a call to `uv_buf_init`), computes $\mathsf{HasPadding}(\texttt{uv_buf_t}) = \texttt{false}$ on the analysis platform, and emits no diagnostics. This is a correct true-negative result that demonstrates the checker's platform-sensitive layout computation.

raylib: value-semantic API with confirmed padding and defensive initialization. All five annotated entry points (`GetRayCollisionSphere`, `GetRayCollisionBox`, `GetRayCollisionMesh`, `GetRayCollisionTriangle`, `GetRayCollisionQuad`) return `RayCollision` by value. The struct layout is:

Field	Type	Offset
`hit`	`bool`	0
(padding)	–	1–3
`distance`	`float`	4
`point`	`Vector3`	8
`normal`	`Vector3`	20
`sizeof(RayCollision)`		32 bytes, 3 bytes padding

The checker extracts 20 E_B events across 29 translation units, identifying `RayCollision` as a padded type at 8 candidate return sites. However, inspection of the raylib implementation reveals that each function zero-initializes its return value before populating fields, satisfying the whole-object initialization predicate. Consequently, all 8 `RayCollision` return events are suppressed ($N_{\text{sup}} = 8$) and no diagnostics are emitted. The remaining 12 events involve `Ray` and `Vector3` parameters, which contain no padding and produce $\mathsf{HasPadding} = \texttt{false}$ results.

This outcome validates the suppression path of the checker on production library code: the structural leakage predicate is met, but the library's defensive initialization practice correctly prevents a diagnostic from being emitted.

Chipmunk2D: confirmed field-wise initialization without zero-initialization. Two of the three annotated entry points return `cpContactPointSet` by value. The struct has a 4-byte padding gap between `int count` and `cpVect normal`, the latter requiring 8-byte alignment. Source inspection confirms that both `cpArbiterGetContactPointSet` (in `cpArbiter.c`) and `cpShapesCollide` (in `cpShape.c`) follow the pattern:

```
cpContactPointSet set;          /* automatic storage, uninitialised */
set.count  = ...;               /* field-wise only */
set.normal = ...;
set.points[i].pointA = ...;
return set;                     /* padding gap never cleared */
```

The checker extracts six E_B events, classifies the initialization as `unknown` (the function body is not visible at the call site in the translation units processed), and emits four E2 diagnostics. The third annotated function, `cpShapeGetBB`, returns `cpBB`, four `cpFloat` fields with uniform alignment and no padding, producing two events with HasPadding = `false` and no diagnostics.

A representative emitted diagnostic has the form:

```
warning: padded struct 'cpContactPointSet' returned across trust boundary
note: 4 bytes of padding between 'count' and 'normal'
note: whole-object initialisation not visible at call site (E2 confidence)
```

This is the primary positive finding of the real-world evaluation. The padding gap in `cpContactPointSet` is never addressed by the implementation, and the checker correctly identifies this at E2 confidence across multiple translation units.

8.6.2 Manual Validation of Findings and Suppressions

To verify that the checker's automated classifications are accurate, each emitted diagnostic and each suppressed candidate from the real-world evaluation was manually inspected against the library source. Table 8.5 records the result of this inspection. Each

Project	Function	Kind	Type	Pad?	WO init?	Result	Class
libuv	`uv_buf_init`	ret	`uv_buf_t`	No	N/A	No warning	TN
raylib	`GetRayCollisionSphere`	ret	`RayCollision`	Yes	Yes	Suppressed	CS
raylib	`GetRayCollisionBox`	ret	`RayCollision`	Yes	Yes	Suppressed	CS
raylib	`GetRayCollisionMesh`	ret	`RayCollision`	Yes	Yes	Suppressed	CS
raylib	`GetRayCollisionTriangle`	ret	`RayCollision`	Yes	Yes	Suppressed	CS
raylib	`GetRayCollisionQuad`	ret	`RayCollision`	Yes	Yes	Suppressed	CS
raylib	`GetRayCollisionTriangle`	call	`Ray`	No	N/A	No warning	TN
raylib	`GetRayCollisionTriangle`	call	`Vector3`	No	N/A	No warning	TN
Chipmunk2D	`cpArbiterGetContactPointSet`	ret	`cpContactPointSet`	Yes	No	E2	TP
Chipmunk2D	`cpShapesCollide`	ret	`cpContactPointSet`	Yes	No	E2	TP
Chipmunk2D	`cpShapeGetBB`	ret	`cpBB`	No	N/A	No warning	TN

Table 8.5: Manual validation of emitted diagnostics and suppressed candidates from the real-world evaluation. *Pad?* and *WO init?* are determined by source inspection. *Class*: TP = true positive; TN = true negative; CS = correct suppression; FP = false positive. Recurrent findings across multiple translation units are represented once per distinct function-type pair and remain reflected in the aggregate counts of Table 8.2.

row corresponds to one distinct function-type pair; where a finding recurs across multiple translation units the recurrence is captured in the aggregate event counts rather than duplicated as rows.

The classification column uses four categories: **TP** (true positive: padding exists and is genuinely not zeroed), **TN** (true negative: no padding, or padding is zeroed, and no diagnostic was emitted), **CS** (correct suppression: padding exists and a diagnostic is correctly suppressed because whole-object initialization is visible), and **FP** (false positive: diagnostic emitted but initialization was present and suppression should have fired).

The validation confirms that all four Chipmunk2D E2 diagnostics are true positives: source inspection of both `cpArbiter.c` and `cpShape.c` confirms that no `memset`, `{0}`, or equivalent zeroing operation precedes the field-wise assignments in either function. The 8 suppressed raylib candidates are all correctly classified as correct suppressions: each `GetRayCollision*` function initializes its local `RayCollision` variable with `= {0}` before populating fields, which the checker correctly recognizes. No false positives were observed among the emitted real-world findings. The false positive in synthetic case 02 arises from a `memset`-based initialization pattern that is not currently recognized as a suppression condition; this is discussed in §8.8.

8.6.3 Cross-Project Observations

Taken together, the four projects support the following observations with respect to the research questions of §8.1.

RQ1 – Event surface. Boundary-transfer events involving by-value aggregate types are absent from libraries with pointer-based API designs (zlib) and rare in libraries where value-semantic APIs are incidental (libuv). They arise frequently in libraries that adopt value semantics as a design principle (raylib, $r_{\text{event}} = 4.0$; Chipmunk2D, $r_{\text{event}} = 2.0$). This suggests that the modeled event surface is closely coupled to API design philosophy rather than to domain or project maturity.

RQ2 – Representation surface. Among projects with non-zero event surfaces, r_{pad} ranges from 0.000 (libuv, where the only by-value type has no padding on the analysis platform) to 0.500 (Chipmunk2D, where one of two encountered types is padded). These values confirm that padding is not universal among by-value aggregate types, and that the checker's layout analysis correctly differentiates padded from non-padded types.

RQ3 – Evidence sensitivity. The suppression mechanism operates correctly in both directions. In raylib, 8 candidate events with confirmed padding are suppressed due to whole-object initialization, and zero diagnostics are emitted, a correct outcome. In Chipmunk2D, no suppression occurs and all padded events produce diagnostics, also a correct outcome. The synthetic benchmark additionally validates the suppression path for the `{0}` pattern, consistent with the lattice definition of §5.6. No E3 diagnostics were observed in the evaluated real-world projects because the checker operates at translation-unit scope: when a struct is returned from a function, the initialization behavior of the callee is not visible without interprocedural analysis, so padded return values are conservatively classified at E2 confidence.

RQ4 – Design implications. The empirical results suggest that mature C systems libraries frequently employ pointer-based or opaque-handle APIs that structurally avoid by-value aggregate transfer. Where value-semantic APIs are adopted, most commonly in graphics, physics, and game-development libraries, the leakage class is structurally present. Whether the risk materializes depends on whether the implementation includes defensive zero-initialization at construction sites, as observed in raylib but absent in Chipmunk2D.

8.7 Performance

Evaluation environment. All measurements were performed on a single machine with the following hardware and software configuration: Intel Core i7-9750H (6 cores, 12 threads via Hyper-Threading, base clock 2.60 GHz), 16 GiB RAM, Arch Linux with kernel 6.18.13-arch1-1, LLVM/Clang 21.1.8, CodeChecker 6.27.3. Analyses were run with

Project	TUs	Baseline (s)	Checker (s)	Overhead
zlib	37	3.61	4.44	+23.0%
libuv	459	40.26	61.35	+52.4%
raylib	29	3.74	5.17	+38.2%
Chipmunk2D	92	10.02	12.01	+19.9%

Table 8.6: Wall-clock analysis times and checker overhead per project. Measurements conducted on an Intel Core i7-9750H system under CPU frequency throttling; see text for caveats on absolute values.

`--jobs 4` parallelism. During the measurements the CPU frequency governor had scaled the clock to approximately 22% of nominal frequency; absolute wall-clock times therefore reflect a power-constrained environment and should not be interpreted as worst-case or representative-of-production figures. Relative overhead percentages remain valid since baseline and checker runs were executed under identical conditions on the same machine.

Results. Table 8.6 reports wall-clock times for both the baseline run (all checks disabled, plugin not loaded) and the checker-enabled run, alongside the computed overhead.

Overhead ranges from approximately 20% (Chipmunk2D) to 52% (libuv). The elevated overhead for libuv is consistent with its large translation unit count (459 TUs): the checker's AST traversal cost is incurred per translation unit, while the number of relevant boundary events remains small relative to the total analysis volume. For smaller projects (zlib, raylib, Chipmunk2D) the overhead is more moderate. In all cases the analysis remains fully incremental within the CodeChecker pipeline and adds no additional inter-procedural passes. A controlled performance study under stable clock frequency, with memory profiling and per-TU breakdown, is left as future work.

8.8 Threats to Validity

Boundary representativeness. Manually annotated boundary interfaces approximate domain transitions but do not guarantee that all security-relevant boundaries are covered. Boundary inference is treated as future work (§9.2). The current evaluation therefore characterizes checker behavior under an explicit boundary model, isolating representation-level reasoning from the separate problem of boundary inference.

Initialization approximation. The structural heuristic $\widehat{\mathrm{Init}}(o)$ may produce false negatives when whole-object clearing occurs indirectly through helper functions not visible in the analyzed translation unit, and false positives when the `{0}` pattern is present but does not cover all padding bytes under edge cases of the C standard [5, 25]. The synthetic

suite also revealed a current implementation limitation: benchmark case 02 does not yet recognize the `memset` pattern as a suppression condition, producing a false positive in the controlled setting. No analogous false positives were observed in the evaluated real-world findings. This behavior is documented in §9.1 and is treated as a staged extension.

Translation-unit scope and layout visibility. Layout computation requires the full record definition to be visible in the analyzed translation unit. Opaque forward declarations and cross-translation-unit definitions can cause the checker to conservatively skip events rather than misclassify them. This affects both event extraction and padding detection [6].

Evidence classification. The E2 versus E3 distinction requires that the initialization pattern at the call site is structurally visible. When the caller obtains a struct from a function return value, the internal initialization pattern of the callee is not visible without interprocedural analysis, and the checker conservatively assigns E2. All four Chipmunk2D diagnostics are therefore classified E2 rather than E3, even though source inspection confirms a field-wise pattern. Interprocedural initialization tracking is left as future work.

Dataset scope. The selected projects span compression, asynchronous I/O, game graphics, and 2D physics. While this provides coverage of both the pointer-based and value-semantic API conventions, results may differ in application codebases, embedded systems, or protocol implementations where aggregate-by-value interfaces or ad-hoc serialization are more prevalent.

Performance measurement. Timing measurements were conducted under CPU frequency throttling (approximately 22% of nominal clock speed), which inflates absolute wall-clock times. Overhead percentages are internally consistent since both baseline and checker runs experienced identical throttling conditions, but should not be taken as representative of production analysis throughput. Memory profiling and per-TU overhead breakdown are left as future work.

External validity. The reported counts and rates reflect the selected benchmark suite and four mature C libraries under explicit boundary annotation. Broader external validity would require analysis of a larger and more diverse corpus, including application codebases and projects with heavier reliance on aggregate-by-value interfaces.

8.9 Summary

Despite the limitations described above, the evaluation demonstrates that the proposed structural predicate can be operationalized in a Clang-based analysis and applied to real-world C libraries. Manual inspection confirmed the correctness of the emitted real-world diagnostics and of the inspected suppressions within the evaluated corpus; no false positives were observed among the emitted real-world findings, with the sole observed false positive arising in the synthetic `memset` case. Just as importantly, the null results are themselves explanatory: pointer-based APIs structurally avoid the event surface, whereas value-semantic APIs expose it and then rise or fall on initialization discipline. Within that contrastive setting, the checker distinguishes platform-dependent true negatives, defensively initialized padded transfers, and unsuppressed candidates for representation-level leakage.

Chapter 9

Limitations and Future Work

This thesis evaluates the expressive power of compiler-visible structural semantics for modeling a restricted class of trust-boundary information leakage. The modeling and implementation intentionally restrict scope to explicit boundary annotations, translation-unit analysis, and conservative initialization heuristics [8, 5, 25]. This chapter documents the known limitations of the current prototype and describes the natural extensions that follow from its design constraints.

9.1 Known Limitations of the Current Prototype

The following limitations are specific to the current implementation rather than fundamental constraints of the modeling framework. Each represents a targeted gap that could be addressed as a staged extension.

Incomplete `memset` suppression recognition. Benchmark case 02 in the synthetic validation suite (Appendix A) reveals that the checker does not yet recognize `memset(&obj, 0, sizeof obj)` as a whole-object initialization pattern for suppression purposes. The `{0}` initialization idiom is correctly recognized, but the `memset` path is not, resulting in a false positive for that pattern. This is a targeted implementation gap rather than a modeling limitation; extending the initialization heuristic to cover `memset` is a straightforward staged extension.

Translation-unit-scoped initialization evidence. The checker approximates whole-object initialization using syntactically visible patterns in the analyzed translation unit. When a struct is returned from a function defined in a separate translation unit, the internal initialization behavior of that function is not visible, and the checker conservatively

assigns E2 rather than E3 confidence. This affects, for example, all four Chipmunk2D diagnostics, which are classified E2 even though source inspection confirms a field-wise pattern. Interprocedural initialization tracking is described as a future direction in §9.3.

Annotation-dependent boundary set. The prototype requires manually annotated trust boundaries. The quality of results therefore depends directly on the completeness and accuracy of the annotation set. Boundary inference, treated as a separate future direction in §9.2, is necessary before the approach can operate without human-provided boundary specifications.

9.2 Automatic Boundary Inference

The present work models trust boundaries through explicit source-level annotations. This isolates the structural semantic question from boundary discovery. However, in realistic systems, trust boundaries are rarely annotated explicitly, and prior work on compartment interface vulnerabilities makes clear that identifying interface-level security boundaries is itself a substantive problem [8].

Future work could investigate boundary inference mechanisms based on:

- Public API visibility and linkage properties.
- Call graph structure and external symbol exposure.
- Data-flow across privilege domains or IPC interfaces.
- Naming conventions and architectural layering.

Such inference would require careful tradeoffs between soundness and usability. Integrating boundary inference with structural semantic reasoning raises a deeper question: whether structural invariants remain meaningfully detectable when boundary identification becomes heuristic rather than explicit.

9.3 Interprocedural Initialization Analysis

The approximation $\widehat{\mathrm{Init}}(o)$ is intentionally conservative and limited to syntactically recognizable initialization patterns. Many real-world programs initialize aggregates indirectly through helper functions, factory routines, or abstraction layers.

Extending the detector to perform limited interprocedural initialization tracking would reduce false positives. Possible directions include:

- Intra-procedural data-flow analysis for full-object writes.
- Tracking initialization guarantees across wrapper functions.
- Recognizing common idioms such as dedicated initialization routines.

Such extensions would increase analysis complexity and potentially shift the tool toward classical data-flow analysis [5, 25, 26]. An important research question is how much additional expressive power is gained relative to the increased computational and implementation cost.

9.4 Pointer-Based Transfers

The present model excludes pointer and reference passing because such transfers do not necessarily imply direct observation of the full object representation at the interface boundary. In many C libraries, structured data crosses boundaries through pointers or opaque handles.

Modeling these cases would require:

- Alias analysis to determine when pointed-to objects are copied or serialized.
- Recognition of serialization patterns such as buffer writes and network sends.
- Integration with interprocedural call-graph reasoning.

This extension would blur the distinction between structural semantic modeling and classical information-flow analysis [25, 22, 28]. Investigating how far structural reasoning can extend before requiring full alias modeling remains an open research direction.

9.5 Configuration-Level Semantic Misconfigurations

While this thesis focuses on representation-level leakage via padding, the broader definition of security semantic misconfiguration introduced in Chapter 4 encompasses additional classes. Future work could extend the modeling framework to capture:

- Compiler configuration flags that alter memory layout or hardening behavior.
- ABI-dependent representation differences across platforms.
- Structure packing directives that trade safety for performance.

- Inconsistent configuration between compilation units.

These phenomena share a common property: they are syntactically valid and standards-compliant, yet may alter security-relevant invariants at the system level [9, 16, 10]. Investigating such configuration-induced semantic misconfigurations would broaden the applicability of compiler-based structural analysis.

9.6 C++ Object Semantics

Although the implementation targets C-heavy codebases, the modeling framework applies to C++ class objects as well. C++ introduces additional complexity, including:

- Implicitly generated constructors.
- Inheritance and base-class layout.
- Virtual dispatch tables.
- Move semantics and copy elision.

Extending the analysis to fully support C++ object models would require deeper integration with language-specific semantics and layout rules [16, 17]. This direction offers opportunities for investigating how modern C++ design patterns interact with structural confidentiality invariants.

9.7 Empirical Scaling

The evaluation in Chapter 8 prioritizes depth of analysis over breadth. A larger-scale empirical study could analyze dozens or hundreds of open-source projects to characterize:

- The distribution of pass-by-value aggregate APIs across domains.
- The frequency of padding in public data structures.
- Correlations between coding conventions and structural leakage risk.

Such a study would strengthen external validity and provide statistical grounding for conclusions about API design practices.

9.8 Formal Soundness

The current framework provides an approximation of invariant violations rather than formal guarantees. A formal semantics of record layout and boundary observability could enable:

- Soundness proofs for restricted language subsets.
- Formal comparison between structural modeling and information-flow systems.
- Integration with verified compilation pipelines.

Exploring the theoretical limits of structural invariant modeling within compiler infrastructure represents a deeper research direction connecting programming language theory and practical static analysis [20, 22, 26].

9.9 Extending Beyond Representation-Level Leakage

This thesis focuses on a single, high-confidence CIV subfamily: unintended exposure of object-representation bytes (e.g., padding) across explicitly annotated trust boundaries. A natural next step is to generalize the taxonomy and checker to additional compartment interface vulnerability patterns (e.g., confused-deputy style argument misuse, inconsistent validation across compartments, or state-dependent interface preconditions) as systematized in the CIV literature [8].

9.10 Returning to Cryptographic Misuse Families

The original thesis framing around cryptographic misuse can be re-incorporated by treating cryptographic API usage constraints as compiler-visible semantic properties (e.g., forbidden algorithms/modes, missing zeroization, key/IV lifetime violations) and by mapping these to boundary-relevant contexts where secrets cross privilege or serialization interfaces [1, 2, 3, 4]. This would extend the current prototype from a representation-semantics instantiation to a broader misconfiguration family under the same annotation-driven analysis paradigm.

Chapter 10

Conclusion

This thesis investigated whether compiler-visible structural semantics are sufficient to approximate a restricted class of trust-boundary information leakage in C programs. The work formalized a representation-level confidentiality invariant grounded in object layout and padding semantics [9, 10], defined a structural boundary transfer model, and implemented a prototype detector as a Clang-Tidy module integrated with CodeChecker [6, 7].

The central contribution is the framing of padding-based trust-boundary leakage as a semantic misconfiguration class rather than a syntactic error. The model isolates boundary transfer events involving pass-by-value and return-by-value aggregate transfers and evaluates whether structural information available at the Abstract Syntax Tree level is expressive enough to approximate violations of the defined invariant [11, 14, 15, 8].

The implementation demonstrates that record layout metadata, explicit boundary annotations, and conservative initialization heuristics are sufficient to detect structural leakage candidates without interprocedural or alias analysis [6, 7, 5, 25]. Validation using a synthetic benchmark suite confirms that the operational predicate aligns with the formal model under controlled conditions.

The contrastive evaluation in Chapter 8 does not attempt a corpus-scale prevalence claim. Instead, it shows that the modeled event surface is strongly conditioned by API design: pointer-oriented and opaque-handle interfaces largely avoid by-value representation transfer, whereas value-semantic interfaces expose it, with the observed outcome then determined by initialization discipline. Manual inspection of the evaluated corpus confirmed the correctness of the emitted real-world diagnostics and of the inspected suppressions, while the synthetic benchmark suite exposed the principal current implementation gap in `memset`-based suppression recognition.

The findings therefore illustrate both the capabilities and the limits of compiler-based

structural reasoning. A meaningful class of representation-level confidentiality violations can be approximated using compiler-visible metadata alone, but expressive boundaries emerge when aliasing, interprocedural initialization, or implicit boundary transitions are introduced [5, 25, 22]. The work thus contributes not only a prototype detector but also a characterization of one boundary of structural invariant modeling within mainstream compiler infrastructure.

Finally, the thesis aligns with broader observations from compartmentalization research: once least-privilege boundaries are introduced, interface design becomes security-critical, and even seemingly benign representation artifacts such as padding can create confidentiality hazards if transferred across compartments [8, 18, 19]. More broadly, this thesis argues that security-relevant misconfigurations often arise from semantically valid but structurally hazardous program designs. By embedding invariant-oriented reasoning into compiler tooling, it becomes possible to surface such risks early in development workflows [4, 7]. While the present study focuses on representation-level trust-boundary leakage, the underlying modeling approach opens a path toward broader investigation of configuration-induced semantic misconfigurations within systems programming environments.

Appendix A

Synthetic Benchmark Suite

This appendix documents the synthetic benchmark suite used to validate the structural trust-boundary leakage detector. Each benchmark is a small C program designed to exercise a specific aspect of the detection predicate defined in Chapter 6.

All benchmarks use the explicit boundary annotation mechanism described in Chapter 7 and are distributed with the prototype implementation in the accompanying public repository.

A.1 Benchmark Matrix

Table A.1 summarizes the benchmark cases and their expected diagnostic outcomes.

Case	Scenario	Transfer Type	Expected R
case01_padding_uninit_pass	Padding present, not cleared	Pass-by-value	Warning
case02_padding_memset_pass	Padding present, memset zero	Pass-by-value	No warning
case03_nopadding_pass	No padding in type	Pass-by-value	No warning
case04_padding_uninit_return	Padding present, not cleared	Return-by-value	Warning
case05_packed_pass	Packed struct, no padding	Pass-by-value	No warning
case06_fieldwise_init_pass	Field-wise init only	Pass-by-value	Warning
case07_designated_init_pass	Full-object zero initialization	Pass-by-value	No warning
case08_nested_padding_pass	Nested struct with padding	Pass-by-value	Warning

Table A.1: Synthetic benchmark cases and expected diagnostic behavior.

A.2 Purpose of the Suite

The synthetic benchmarks serve as controlled validation cases for the implementation. They demonstrate that:

- Diagnostics are emitted when padding exists and full-object initialization is not observed.
- Diagnostics are suppressed when padding does not exist.
- Diagnostics are suppressed when full-object zero initialization is syntactically evident.
- The detector behaves consistently for both pass-by-value and return-by-value transfer events.

These benchmarks provide positive and negative controls that validate the operationalization of the structural leakage predicate.

Bibliography

[1] Manuel Egele et al. "An Empirical Study of Cryptographic Misuse in Android Applications". In: *ACM Conference on Computer and Communications Security (CCS)*. Accessed 2026-03-06. 2013. DOI: 10.1145/2508859.2516693. URL: https://megele.io/cryptolint-ccs2013.pdf.

[2] Sazzadur Rahaman et al. "CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects". In: *ACM Conference on Computer and Communications Security (CCS)*. Accessed 2026-03-06. 2019. DOI: 10.1145/3319535.3345659. URL: https://yaogroup.cs.vt.edu/papers/CryptoGuardCameraReady.pdf.

[3] Stefan Krüger et al. "CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs". In: *European Conference on Object-Oriented Programming (ECOOP)*. Accessed 2026-03-06. 2018. DOI: 10.4230/LIPIcs.ECOOP.2018.10. URL: https://drops.dagstuhl.de/storage/00lipics/lipics-vol109-ecoop2018/LIPIcs.ECOOP.2018.10/LIPIcs.ECOOP.2018.10.pdf.

[4] Yikang Chen et al. "Towards Precise Reporting of Cryptographic Misuses". In: *Network and Distributed System Security Symposium (NDSS)*. Accessed 2026-03-06. 2024. URL: https://www.ndss-symposium.org/wp-content/uploads/2024-1032-paper.pdf.

[5] Marco Pistoia et al. "A Survey of Static Analysis Methods for Identifying Security Vulnerabilities in Software Systems". In: *IBM Systems Journal* (2007). Accessed 2026-03-06. URL: https://research.ibm.com/publications/a-survey-of-static-analysis-methods-for-identifying-security-vulnerabilities-in-software-systems.

[6] LLVM Project. *Clang Documentation*. https://clang.llvm.org/docs/. Accessed 2026.

[7] Ericsson. *CodeChecker Documentation*. https://codechecker.readthedocs.io/. Accessed 2026.

[8] Hugo Lefeuvre et al. "Assessing the Impact of Interface Vulnerabilities in Compartmentalized Software". In: *Network and Distributed System Security (NDSS) Symposium*. Accessed: 2026. 2023. DOI: 10.14722/ndss.2023.24117.

[9] *Programming Languages – C*. International Organization for Standardization, 2018.

[10] System V Application Binary Interface. *System V Application Binary Interface: AMD64 Architecture Processor Supplement*. https://refspecs.linuxfoundation.org/elf/x86_64-abi-0.99.pdf. Accessed 2026-03-06.

[11] Haehyun Cho, Jinbum Park, Joonwon Kang, et al. "Exploiting Uses of Uninitialized Stack Variables in Linux Kernels to Leak Kernel Pointers". In: *USENIX Workshop on Offensive Technologies (WOOT)*. Accessed 2026-03-06. 2020. URL: https://www.usenix.org/system/files/woot20-paper-cho.pdf.

[12] Microsoft. *CodeQL: Padding Byte Information Disclosure (Windows Driver Query Documentation)*. https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/codeql-windows-driver-padding-byte-information-disclosure. Accessed 2026-03-06.

[13] MathWorks. *Polyspace Bug Finder: Information leak via structure padding*. https://www.mathworks.com/help/bugfinder/ref/informationleakviastructurepadding.html. Accessed 2026-03-06.

[14] Kangjie Lu et al. "UniSan: Proactive Kernel Memory Initialization to Eliminate Data Leakages". In: *ACM Conference on Computer and Communications Security (CCS)*. Accessed 2026-03-06. 2016. DOI: 10.1145/2976749.2978366. URL: https://www.cs.ucr.edu/~csong/ccs16-unisan.pdf.

[15] Sebastian Österlund et al. "kMVX: Detecting Kernel Information Leaks with Multi-variant Execution". In: *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Accessed 2026-03-06. 2019. DOI: 10.1145/3297858.3304054. URL: https://research.vu.nl/files/122357910/KMVX.pdf.

[16] *Programming Languages – C++*. International Organization for Standardization, 2020.

[17] *Itanium C++ ABI*. https://itanium-cxx-abi.github.io/cxx-abi/. Accessed 2026.

[18] SEI CERT. *DCL39-C. Avoid information leakage when passing a structure across a trust boundary*. https://wiki.sei.cmu.edu/confluence/spaces/c/pages/87151978. Accessed 2026. 2023.

[19] SEI CERT. *DCL55-CPP. Avoid information leakage when passing a class object across a trust boundary*. https://wiki.sei.cmu.edu/confluence/spaces/cplusplus/pages/88046453. Accessed 2026. 2023.

[20] Dorothy E. Denning. "A Lattice Model of Secure Information Flow". In: *Communications of the ACM* 19.5 (1976), pp. 236–243.

[21] Joseph A. Goguen and José Meseguer. "Security Policies and Security Models". In: *IEEE Symposium on Security and Privacy*. Introduces noninterference as a formal security property. 1982.

[22] Andrei Sabelfeld and Andrew C. Myers. "Language-Based Information-Flow Security". In: *IEEE Journal on Selected Areas in Communications* 21.1 (2003), pp. 5–19.

[23] Dennis Volpano, Cynthia Irvine, and Geoffrey Smith. "A Sound Type System for Secure Flow Analysis". In: *Journal of Computer Security* 4.3 (1996), pp. 167–187.

[24] Andrew C. Myers. "JFlow: Practical Mostly-Static Information Flow Control". In: *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. Classic language-based information-flow system. 1999.

[25] V. Benjamin Livshits and Monica S. Lam. "Finding Security Vulnerabilities in Java Applications with Static Analysis". In: *USENIX Security Symposium*. Accessed 2026-03-06. 2005. URL: https://www.usenix.org/event/sec05/tech/full_papers/livshits/livshits.pdf.

[26] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs". In: *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. Classic foundation of abstract interpretation. 1977.

[27] LLVM Project. *LLVM Language Reference Manual*. https://llvm.org/docs/LangRef.html. Accessed 2026.

[28] James Newsome and Dawn Song. "Dynamic Taint Analysis for Automatic Detection, Analysis, and Signature Generation of Exploits on Commodity Software". In: *Network and Distributed System Security Symposium (NDSS)*. Accessed 2026-03-06. 2005. URL: https://www.ndss-symposium.org/wp-content/uploads/2017/09/Dynamic-Taint-Analysis-for-Automatic-Detection-Analysis-and-SignatureGeneration-of-Exploits-on-Commodity-Software-Dawn-Song.pdf.

[29] William Enck et al. "TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones". In: *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Accessed 2026-03-06. 2010. URL: https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Enck.pdf.

[30] Carlos Alfaro et al. "Detecting Misuse of Security APIs: A Systematic Review". In: *arXiv* (2023). Survey of security API misuse detection. URL: https://arxiv.org/abs/2306.08869.

[31] Tao Gu et al. "An Empirical Study on API Misuse Bugs in Open Source C Projects". In: *IEEE International Computer Software and Applications Conference (COMPSAC)*. API misuse framing useful for taxonomy positioning. 2019.

[32] Shiqi Ma et al. "CDRep: Automatic Repair of Cryptographic Misuses in Android Applications". In: *IEEE/ACM International Conference on*

Automated Software Engineering (ASE). Repair-oriented approach; useful for remediation discussion. 2016.